

LU DECOMPOSITION

Using Elementary Matrices in Java

A Technical Framework

LinSolve

Version 1.0

2026

LinSolve

Contents

1	Introduction	1
1.1	Why Decompose Matrices?	1
1.2	The Normal Equation in Linear Regression	1
1.2.1	The Computational Problem	1
1.2.2	Where Decompositions Help	2
1.3	What is LU Decomposition?	2
1.3.1	Visual Representation	2
1.3.2	Why LU is Important	2
1.4	Overview of the Elementary Matrix Approach	3
1.4.1	The Elementary Matrix Philosophy	3
1.4.2	The Three Elementary Operations	3
1.4.3	What This Framework Covers	3
1.5	Prerequisites	3
1.6	Guide Structure	4
1.7	Notation Conventions	4
1.8	Getting Started	4
	Package Structure	7
2	Core Matrix Components	1
2.1	Introduction	1
2.2	The Matrix Class	1
2.2.1	Class Implementation	1
2.2.2	Key Design Decisions	4
2.3	The SquareMatrix Class	4
2.3.1	Class Implementation	4
2.3.2	Properties of Square Matrices	5
2.4	The Identity Class	5
2.4.1	Class Implementation	5
2.4.2	Properties of the Identity Matrix	6
2.5	Summary	6
3	The Eij Component - Basis of Matrix Vector Space	9
3.1	Introduction	9
3.2	What is Eij?	9

3.2.1	Examples of Eij	9
3.2.2	Basis Representation Theorem	10
3.2.3	Proof Sketch	10
3.3	The Kronecker Class	10
3.3.1	Class Implementation	10
3.3.2	How the Kronecker Class Works	11
3.4	The Eij Class Implementation	11
3.4.1	Class Implementation	12
3.4.2	Construction Process	13
3.4.3	Example Execution	13
3.5	Why Eij Matters	13
3.5.1	Basis for Arbitrary Matrices	14
3.6	Next Steps	15
4	Operations on Rows as Matrix Products	17
4.1	Introduction	17
4.2	Mathematical Foundation	17
4.3	Row Scaling: $L_i \leftarrow \lambda L_i$	18
4.3.1	Mathematical Definition	18
4.3.2	Example	18
4.3.3	Java Implementation	18
4.4	Row Addition: $L_i \leftarrow L_i + \lambda L_j$	20
4.4.1	Mathematical Definition	20
4.4.2	Example	20
4.4.3	Java Implementation	20
4.5	Row Swapping: $L_i \leftrightarrow L_j$	22
4.5.1	Mathematical Definition	22
4.5.2	Example	22
4.5.3	Java Implementation	22
4.6	Why These Operations are Invertible	24
4.6.1	Proof of Invertibility	24
4.7	Summary	24
4.8	Next Steps	25
5	Forward Elimination (Gauss Pivot)	27
5.1	Introduction	27
5.2	The Algorithm	27
5.2.1	Assumptions	27
5.2.2	Step-by-Step Description	27
5.2.3	Mathematical Formulation	28
5.3	Accumulation of Elementary Matrices	28
5.4	Java Implementation	28
5.4.1	The ApplyingGaussElimination Class	28
5.5	Step-by-Step Example	31
5.5.1	Step 1: Column 0 ($j = 0$)	31

5.5.2	Step 2: Column 1 ($j = 1$)	32
5.5.3	Step 3: Column 2 ($j = 2$)	33
5.5.4	Result	33
5.6	Formatting the Result	33
5.7	Summary	34
5.8	Next Steps	34
6	Backward Elimination (Back Substitution)	35
6.1	Introduction	35
6.2	The Algorithm	35
6.2.1	Mathematical Formulation	35
6.2.2	Accumulation	36
6.3	The Complete Inverse	36
6.4	Java Implementation	36
6.4.1	The ApplyingBackSubstitution Class	36
6.5	Step-by-Step Example (Continued)	41
6.5.1	Step 1: Column 2 ($j = 2$)	41
6.5.2	Step 2: Column 1 ($j = 1$)	41
6.5.3	Result	42
6.6	Why This Works	42
6.6.1	Proof	42
6.7	Summary	42
6.8	Next Steps	43
7	Extracting L and U via Triangular Inversion	45
7.1	Introduction	45
7.2	The Relationship Between A^{-1} , L, and U	45
7.3	Extracting L and U	46
7.3.1	Key Insight	46
7.3.2	Why Triangular Matrices Are Easy to Invert	46
7.4	Inverting Lower Triangular Matrices	47
7.4.1	Algorithm	47
7.4.2	Java Implementation	47
7.5	Inverting Upper Triangular Matrices	50
7.5.1	Algorithm	50
7.5.2	Java Implementation	50
7.6	Complete LU Reconstruction	53
7.6.1	The FormulatingLUComponents Class	53
7.7	Verification	56
7.7.1	Example Verification	56
7.8	Summary	57
7.9	Next Steps	57

8	Complete LU Decomposition Framework	59
8.1	Introduction	59
8.2	The ConstructingLUComponents Class	59
8.2.1	Class Implementation	59
8.3	Complete Workflow Diagram	61
8.4	Complete Example	61
8.4.1	Input Matrix	61
8.4.2	Step-by-Step Execution	62
8.5	Performance Considerations	64
8.5.1	Time Complexity	64
8.5.2	Space Complexity	64
8.5.3	Numerical Stability	64
8.6	Testing with Random Matrices	64
8.7	Summary	65
8.8	Next Steps	65
9	Perspectives - PLU Decomposition	67
9.1	Introduction	67
9.2	Why Pivoting is Necessary	67
9.2.1	The Problem with Zero Pivots	67
9.2.2	The Problem with Small Pivots	67
9.2.3	Solution: Row Swapping	68
9.3	The Permutation Matrix P	68
9.3.1	Properties of Permutation Matrices	68
9.3.2	Construction from $L_i_swap_L_j$	68
9.4	PLU Algorithm with Partial Pivoting	68
9.4.1	Algorithm Description	68
9.4.2	Mathematical Formulation	69
9.4.3	Permutation Accumulation	69
9.5	Java Implementation Extension	69
9.5.1	Modified Gauss Elimination with Pivoting	69
9.5.2	Complete PLU Class Structure	70
9.6	Example: PLU Decomposition	72
9.6.1	Matrix Requiring Pivoting	72
9.6.2	Step 1: Column 0	72
9.6.3	Step 2: Column 1	73
9.6.4	Result	73
9.7	Solving Linear Systems with PLU	74
9.8	Partial vs. Complete Pivoting	75
9.9	Summary	75
9.10	Next Steps	75

Listings

2.1	Matrix.java	2
2.2	SquareMatrix.java	4
2.3	Identity.java	5
3.1	Kronecker.java	10
3.2	Eij.java	12
4.1	<i>Li_lamda_Li.java</i>	18
4.2	<i>Li_plus_lamda_Lj.java</i>	20
4.3	<i>Li_swap_Lj.java</i>	22
5.1	ApplyingGaussElimination.java	28
5.2	Formatting the result	33
6.1	ApplyingBackSubstitution.java	36
7.1	InvertingLowerMatrix.java	47
7.2	InvertingUpperMatrix.java	50
7.3	FormulatingLUComponents.java	54
8.1	ConstructingLUComponents.java	59
8.2	Testing with random matrices	64
9.1	ApplyingGaussElimination with pivoting	69
9.2	PLUDecomposition.java skeleton	70
9.3	Solving with PLU	74

Chapter 1

Introduction

1.1 Why Decompose Matrices?

Solving systems of linear equations is at the heart of scientific computing, machine learning, and engineering. Consider the canonical problem:

$$A\mathbf{x} = \mathbf{b} \tag{1.1}$$

Where A is an $n \times n$ matrix, $\mathbf{x}$ is the unknown vector, and $\mathbf{b}$ is the known right-hand side.

The direct solution using Cramer's rule requires $O(n!)$ operations — impossible for large n . Matrix decompositions reduce this to $O(n^3)$.

1.2 The Normal Equation in Linear Regression

In machine learning, linear regression finds the best-fit line through data. Given a design matrix X (size $m \times n$) and target vector $\mathbf{y}$ (size m), the goal is to minimize:

$$\min_{\boldsymbol{\beta}} \|X\boldsymbol{\beta} - \mathbf{y}\|_2^2 \tag{1.2}$$

The analytical solution is given by the **Normal Equation**:

$$\boxed{(X^T X)\boldsymbol{\beta} = X^T \mathbf{y}} \tag{1.3}$$

Or equivalently:

$$\boldsymbol{\beta} = (X^T X)^{-1} X^T \mathbf{y} \tag{1.4}$$

1.2.1 The Computational Problem

The Normal Equation requires:

1. Computing $X^T X$ — $O(mn^2)$ operations

2. Inverting $X^T X$ — $O(n^3)$ operations

For large n (many features), this becomes prohibitively expensive or numerically unstable.

1.2.2 Where Decompositions Help

Matrix decompositions provide **numerically stable** and **computationally efficient** alternatives:

Decomposition	Formula	Use in Regression
LU	$A = L \times U$	Solve $(X^T X)\boldsymbol{\beta} = X^T \mathbf{y}$
QR	$X = Q \times R$	Direct solution without forming $X^T X$
SVD	$X = U \Sigma V^T$	Ridge regression, PCA, pseudoinverse

1.3 What is LU Decomposition?

LU decomposition factors a square matrix A into:

$$A = L \times U \quad (1.5)$$

Where:

- L is **lower triangular** (all entries above diagonal are zero)
- U is **upper triangular** (all entries below diagonal are zero)
- Typically, L has ones on its diagonal (unit lower triangular)

1.3.1 Visual Representation

For a 4×4 matrix:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ l_{21} & 1 & 0 & 0 \\ l_{31} & l_{32} & 1 & 0 \\ l_{41} & l_{42} & l_{43} & 1 \end{bmatrix} \times \begin{bmatrix} u_{11} & u_{12} & u_{13} & u_{14} \\ 0 & u_{22} & u_{23} & u_{24} \\ 0 & 0 & u_{33} & u_{34} \\ 0 & 0 & 0 & u_{44} \end{bmatrix}$$

1.3.2 Why LU is Important

1. **Solve multiple right-hand sides:** Once $A = LU$ is computed, solving $A\mathbf{x} = \mathbf{b}$ for different $\mathbf{b}$ becomes trivial:
 - Forward substitution: Solve $L\mathbf{y} = \mathbf{b}$ — $O(n^2)$
 - Back substitution: Solve $U\mathbf{x} = \mathbf{y}$ — $O(n^2)$
2. **Determinant calculation:** $\det(A) = \prod_{i=1}^n u_{ii}$
3. **Matrix inversion:** $A^{-1} = (LU)^{-1} = U^{-1}L^{-1}$
4. **Foundation for other decompositions:** LU is the basis for QR, Cholesky, and SVD algorithms

1.4 Overview of the Elementary Matrix Approach

Traditional LU implementations use nested loops with scalar operations. **This hides the linear algebra.**

1.4.1 The Elementary Matrix Philosophy

Our approach treats **every operation as a matrix multiplication**:

$$E \times A \rightarrow A_{\text{new}} \tag{1.6}$$

Where E is an **elementary matrix** — the identity matrix plus a single modification.

1.4.2 The Three Elementary Operations

1. **Row scaling**: Li_lamda_Li — multiply row i by λ
2. **Row addition**: Li_plus_lamda_Lj — add $\lambda \times$ row j to row i
3. **Row swapping**: Li_swap_Lj — exchange rows i and j

1.4.3 What This Framework Covers

- Complete implementation of matrix and elementary matrix classes in Java
- Step-by-step construction of LU decomposition using elementary operations
- Forward elimination (Gauss elimination) to obtain upper triangular matrix
- Backward elimination (back substitution) to obtain the inverse
- Extraction of L and U via triangular matrix inversion
- Extension to PLU (permutation + LU) for matrices requiring pivoting
- Complete source code project included with this framework

1.5 Prerequisites

- Basic Java programming (classes, loops, arrays)
- Basic linear algebra (matrix multiplication, triangular matrices)
- No external libraries required — everything implemented from scratch

1.6 Guide Structure

Part	Content
Part I	Foundations — Matrix, SquareMatrix, Identity, Eij, Kronecker
Part II	Basic Matrix Operations — Addition, multiplication, scalar multiplication
Part III	Elementary Matrices — Li_lamda_Li, Li_plus_lamda_Lj, Li_swap_Lj
Part IV	LU Decomposition — Forward elimination, backward elimination, inverse computation
Part V	Complete Implementation — Extracting L and U via triangular inversion
Part VI	Perspectives — PLU decomposition with pivoting
Appendices	Complete source code, mathematical prerequisites, glossary

1.7 Notation Conventions

Throughout this document, we use the following conventions:

- Matrices are denoted by capital letters: A , L , U , P
- Vectors are denoted by bold lowercase letters: $\mathbf{x}$, $\mathbf{b}$
- Scalars are denoted by lowercase letters: λ , a_{ij}
- Indices start at 0 (Java convention) for both mathematical formulas and code
- I_n denotes the $n \times n$ identity matrix
- E_{ij} denotes the basis matrix with 1 at position (i, j) and 0 elsewhere

1.8 Getting Started

The complete source code accompanying this framework is organized in the following package structure:

List of Figures

8.1 LU Decomposition Workflow	62
---	----

Package Structure

```
com.linsolve.decompositions.lu/  
|-- components/  
|   |-- Matrix.java  
|   |-- SquareMatrix.java  
|   |-- Identity.java  
|   |-- Eij.java  
|   `-- Kronecker.java  
|-- components.basicoperations/  
|   |-- AddingMatrices.java  
|   |-- MultiplyingMatrices.java  
|   `-- MultiplyingMatrixByScalar.java  
|-- components.elementaryoperations/  
|   |-- Li_lamda_Li.java  
|   |-- Li_plus_lamda_Lj.java  
|   `-- Li_swap_Lj.java  
|-- components.helper/  
|   |-- GeneratingRandomMatrix.java  
|   |-- GeneratingRandomLowerMatrix.java  
|   |-- GeneratingRandomUpperMatrix.java  
|   |-- InvertingLowerMatrix.java  
|   `-- InvertingUpperMatrix.java  
|-- operations/  
|   |-- ApplyingGaussElimination.java  
|   |-- ApplyingBackSubstitution.java  
|   |-- FormulatingLUComponents.java  
|   `-- ConstructingLUComponents.java  
`-- exceptions.components/  
    |-- MatrixConstructionEnum.java  
    `-- MatrixConstructionException_001.java
```


Chapter 2

Core Matrix Components

2.1 Introduction

The foundation of any matrix computation library is a robust representation of matrices. This chapter presents the core classes that form the building blocks of our LU decomposition framework.

All matrix classes store data in a two-dimensional array of `Double` objects, allowing for:

- Dynamic sizing at runtime
- Null value representation (if needed)
- Easy integration with other numerical methods

2.2 The Matrix Class

The `Matrix` class is the fundamental building block. It stores a rectangular array of `Double` values with dimensions $m \times n$.

Definition

A **Matrix** of size $m \times n$ is a rectangular array of numbers arranged in m rows and n columns:

$$A = \begin{bmatrix} a_{00} & a_{01} & \cdots & a_{0,n-1} \\ a_{10} & a_{11} & \cdots & a_{1,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m-1,0} & a_{m-1,1} & \cdots & a_{m-1,n-1} \end{bmatrix}$$

The entry in row i and column j is denoted a_{ij} .

2.2.1 Class Implementation

```
1 package com.linsolve.decompositions.lu.components;
2
3 public class Matrix {
4     protected Integer m;        // Number of rows
5     protected Integer n;        // Number of columns
6     protected Double[][] t;     // The data array
7
8     // Default constructor
9     public Matrix() {
10         super();
11     }
12
13     // Constructor with dimensions
14     public Matrix(Integer m, Integer n) {
15         super();
16         this.m = m;
17         this.n = n;
18         this.t = new Double[m][n];
19     }
20
21     // Constructor with data
22     public Matrix(Integer m, Integer n, Double[][] t) {
23         super();
24         this.m = m;
25         this.n = n;
26         if (t.length != m || t[0].length != n) {
27             try {
28                 throw new MatrixConstructionException_001(
29                     MatrixConstructionEnum.SizeDoubleArrayDoNotMatch
30                 );
31             } catch (MatrixConstructionException_001 e) {
32                 System.out.println(e.getMessage());
33             }
34         } else {
35             this.t = t;
36         }
37     }
38
39     // Display methods
40     public static void display(Double[][] t) {
41         for (int k = 0; k < t.length; k++) {
42             System.out.print("[");
43             for (int l = 0; l < t[0].length - 1; l++) {
44                 System.out.print(t[k][l]);
45                 System.out.print("|| ");
46             }
47             System.out.print(t[k][t[0].length - 1]);
```

```
48         System.out.print("]");
49         System.out.println();
50     }
51 }
52
53 public static void displayVector(Double[] v) {
54     System.out.print("[");
55     for (int l = 0; l < v.length - 1; l++) {
56         System.out.print(v[l]);
57         System.out.print("||");
58     }
59     System.out.print(v[v.length - 1]);
60     System.out.print("]");
61     System.out.println();
62 }
63
64 public static void displayMatrix(Matrix matrix) {
65     display(matrix.getT());
66 }
67
68 // Getters and setters
69 public Integer getM() { return m; }
70 public void setM(Integer m) { this.m = m; }
71 public Integer getN() { return n; }
72 public void setN(Integer n) { this.n = n; }
73 public Double[][] getT() { return t; }
74 public void setT(Double[][] t) { this.t = t; }
75 }
```

Listing 2.1: Matrix.java

2.2.2 Key Design Decisions

Important

The design of the `Matrix` class incorporates several important decisions:

- `protected` fields allow subclasses (`SquareMatrix`, `Identity`) to access data directly without getter overhead
- `Double` (object) rather than `double` (primitive) allows null representation and easier integration with generic collections
- Validation in the constructor prevents dimension mismatches between specified dimensions and actual array sizes
- Static display methods provide convenient output formatting for debugging and verification

2.3 The SquareMatrix Class

`SquareMatrix` extends `Matrix` and enforces the condition $m = n$. Square matrices are essential for LU decomposition, eigenvalue problems, and many other linear algebra operations.

Definition

A **Square Matrix** is a matrix with the same number of rows and columns: $m = n$. The set of all $n \times n$ square matrices is denoted $\mathbb{R}^{n \times n}$.

2.3.1 Class Implementation

```
1 package com.linsolve.decompositions.lu.components;
2
3 public class SquareMatrix extends Matrix {
4     protected Integer n;
5
6     public SquareMatrix() {
7         super();
8     }
9
10    public SquareMatrix(Integer n) {
11        super(n, n);
12        this.n = n;
13        this.m = n;
14        this.t = new Double[n][n];
15
16        // Initialize to zeros
17        for (int i = 0; i < this.t.length; i++) {
```

```

18         for (int j = 0; j < this.t[0].length; j++) {
19             t[i][j] = 0.0;
20         }
21     }
22 }
23
24 public Integer getN() { return n; }
25 public void setN(Integer n) { this.n = n; }
26 }

```

Listing 2.2: SquareMatrix.java

2.3.2 Properties of Square Matrices

Square matrices have special properties that make them particularly important:

Property	Description
Determinant	$\det(A)$ exists only for square matrices
Inverse	A^{-1} exists only if $\det(A) \neq 0$
Eigenvalues	Characteristic polynomial $\det(A - \lambda I) = 0$ defined
Trace	$\text{tr}(A) = \sum_{i=0}^{n-1} a_{ii}$

2.4 The Identity Class

Identity represents the identity matrix I_n , where $I_{ij} = 1$ if $i = j$, and 0 otherwise.

Definition

The **Identity Matrix** I_n is the $n \times n$ square matrix with ones on the main diagonal and zeros elsewhere:

$$I_n = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix}$$

The identity matrix satisfies $I_n A = A I_n = A$ for any $n \times n$ matrix A .

2.4.1 Class Implementation

```

1 package com.linsolve.decompositions.lu.components;
2
3 public class Identity extends SquareMatrix {
4     private Integer n;
5     private Kronecker kroneckerij;
6
7     public Identity(Integer n) {

```

```

8      super(n);
9      this.n = n;
10     for (int i = 0; i < this.t.length; i++) {
11         for (int j = 0; j < this.t[0].length; j++) {
12             this.kroneckerij = new Kronecker(i, i);
13             this.kroneckerij.applyKronecker(i, j);
14             this.t[i][j] = this.kroneckerij.getKroneckerij();
15         }
16     }
17 }

18
19 public Integer getN() { return n; }
20 public void setN(Integer n) { this.n = n; }
21 public Kronecker getKroneckerij() { return kroneckerij; }
22 public void setKroneckerij(Kronecker kroneckerij) {
23     this.kroneckerij = kroneckerij;
24 }
25
26 public static void main(String[] args) {
27     Identity id = new Identity(5);
28     Matrix.displayMatrix(id);
29 }
30 }

```

Listing 2.3: Identity.java

2.4.2 Properties of the Identity Matrix

Important

The identity matrix is the neutral element for matrix multiplication:

$$I_n A = A I_n = A$$

Key properties:

- I_n is symmetric: $I_n^T = I_n$
- I_n is its own inverse: $I_n^{-1} = I_n$
- $\det(I_n) = 1$
- All eigenvalues of I_n are 1

2.5 Summary

This chapter introduced the foundational matrix classes:

| **Matrix** | General $m \times n$ matrices | Basic storage and display | | **SquareMatrix** | $n \times n$ matrices | Enforces square dimensions | | **Identity** | Identity matrix I_n | Neutral element for multiplication |

These classes provide the data structures needed for the elementary matrix operations and LU decomposition algorithms presented in subsequent chapters.

In the next chapter, we introduce the **Eij** component — the basis of the matrix vector space that enables the construction of arbitrary matrices and elementary operations.

Chapter 3

The Eij Component - Basis of Matrix Vector Space

3.1 Introduction

The key insight of this framework is the use of **Eij** as the fundamental building block for all matrices. Just as any vector can be expressed as a linear combination of basis vectors, any matrix can be expressed as a linear combination of **Eij** matrices.

This chapter introduces the **Eij** matrix, the **Kronecker** delta function that implements it, and demonstrates how arbitrary matrices can be constructed from these atomic components.

3.2 What is Eij?

Definition

The matrix E_{ij} (also denoted $e_i e_j^T$) has a 1 at position (i, j) and zeros everywhere else:

$$(E_{ij})_{kl} = \begin{cases} 1 & \text{if } k = i \text{ and } l = j \\ 0 & \text{otherwise} \end{cases}$$

For an $m \times n$ matrix space, the set $\{E_{ij} : 0 \leq i < m, 0 \leq j < n\}$ forms a basis.

3.2.1 Examples of Eij

For 3×3 matrices, several examples illustrate the concept:

$$\begin{aligned} E_{00} &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, & E_{01} &= \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, & E_{11} &= \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \\ E_{12} &= \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, & E_{21} &= \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, & E_{22} &= \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

3.2.2 Basis Representation Theorem

Theorem

Any $m \times n$ matrix A can be uniquely expressed as a linear combination of E_{ij} matrices:

$$A = \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} a_{ij} E_{ij}$$

where a_{ij} are the entries of A .

3.2.3 Proof Sketch

The theorem follows directly from the definition of E_{ij} :

1. Each E_{ij} has a 1 only at position (i, j)
2. Multiplying E_{ij} by a_{ij} places a_{ij} at position (i, j)
3. Summing over all i, j reconstructs the original matrix

3.3 The Kronecker Class

The `Kronecker` class implements the Kronecker delta function, which is essential for constructing E_{ij} matrices.

Definition

The **Kronecker Delta** δ_{ij} is defined as:

$$\delta_{ij} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

For our purposes, we extend this to a function of four indices:

$$\delta_{(i,j),(k,l)} = \begin{cases} 1 & \text{if } i = k \text{ and } j = l \\ 0 & \text{otherwise} \end{cases}$$

3.3.1 Class Implementation

```

1 package com.linsolve.decompositions.lu.components;
2
3 public class Kronecker {
4     private Integer i;
5     private Integer j;
6     private Double kroneckerij;

```

```

7
8     public Kronecker() {
9         super();
10    }
11
12    public Kronecker(Integer i, Integer j) {
13        super();
14        this.i = i;
15        this.j = j;
16        this.kroneckerij = 0.0;
17    }
18
19    public void applyKronecker(int k, int l) {
20        if (i == k && j == l) {
21            this.kroneckerij = 1.0;
22        } else {
23            this.kroneckerij = 0.0;
24        }
25    }
26
27    public Integer getI() { return i; }
28    public void setI(Integer i) { this.i = i; }
29    public Integer getJ() { return j; }
30    public void setJ(Integer j) { this.j = j; }
31    public Double getKroneckerij() { return kroneckerij; }
32    public void setKroneckerij(Double kroneckerij) {
33        this.kroneckerij = kroneckerij;
34    }
35 }

```

Listing 3.1: Kronecker.java

3.3.2 How the Kronecker Class Works

The `Kronecker` class stores a target position (i, j) . When `applyKronecker(k, l)` is called, it returns 1 if (k, l) matches the stored position, and 0 otherwise.

Important

This implementation allows the same `Kronecker` object to be reused for generating entire rows or columns of E_{ij} matrices.

3.4 The Eij Class Implementation

The `Eij` class extends `Matrix` and uses `Kronecker` to generate a matrix with a single 1 at position (i, j) .

3.4.1 Class Implementation

```

1 package com.linsolve.decompositions.lu.components;
2
3 public class Eij extends Matrix {
4     private Integer i;
5     private Integer j;
6     private Kronecker kroneckerij;
7
8     public Eij(Integer i, Integer j, Integer m, Integer n) {
9         super(m, n);
10        this.i = i;
11        this.j = j;
12        this.kroneckerij = new Kronecker(i, j);
13
14        // Generate the matrix by applying Kronecker delta at each
15        // position
16        for (int k = 0; k < super.m; k++) {
17            for (int l = 0; l < super.n; l++) {
18                this.kroneckerij.applyKronecker(k, l);
19                this.t[k][l] = this.kroneckerij.getKroneckerij();
20            }
21        }
22
23        public Integer getI() { return i; }
24        public void setI(Integer i) { this.i = i; }
25        public Integer getJ() { return j; }
26        public void setJ(Integer j) { this.j = j; }
27        public Kronecker getKroneckerij() { return kroneckerij; }
28        public void setKroneckerij(Kronecker kroneckerij) {
29            this.kroneckerij = kroneckerij;
30        }
31
32        public static void main(String[] args) {
33            // Demonstrate various Eij matrices
34            Eij eij = new Eij(2, 3, 5, 5);
35            Eij eij_001 = new Eij(2, 2, 5, 5);
36            Eij eij_002 = new Eij(0, 4, 5, 5);
37            Eij eij_003 = new Eij(1, 3, 5, 5);
38            Eij eij_004 = new Eij(0, 0, 5, 5);
39
40            System.out.println("E(2,3):");
41            Matrix.displayMatrix(eij);
42
43            System.out.println("\nE(2,2):");
44            Matrix.displayMatrix(eij_001);
45

```

```

46      System.out.println("\nE(0,4):");
47      Matrix.displayMatrix(eij_002);
48
49      System.out.println("\nE(1,3):");
50      Matrix.displayMatrix(eij_003);
51
52      System.out.println("\nE(0,0):");
53      Matrix.displayMatrix(eij_004);
54  }
55 }

```

Listing 3.2: Eij.java

3.4.2 Construction Process

The Eij constructor follows these steps:

1. Calls `super(m, n)` to create an $m \times n$ zero matrix
2. Stores the target indices i and j
3. Creates a `Kronecker` object with the target position
4. Iterates over all matrix positions (k, l)
5. For each position, applies `applyKronecker(k, l)`
6. Places the resulting value (1 or 0) in the matrix

3.4.3 Example Execution

For `Eij(1, 2, 3, 3)`, the generation process yields:

$$\begin{bmatrix} \delta_{(1,2),(0,0)} & \delta_{(1,2),(0,1)} & \delta_{(1,2),(0,2)} \\ \delta_{(1,2),(1,0)} & \delta_{(1,2),(1,1)} & \delta_{(1,2),(1,2)} \\ \delta_{(1,2),(2,0)} & \delta_{(1,2),(2,1)} & \delta_{(1,2),(2,2)} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

3.5 Why Eij Matters

The E_{ij} matrix is the atomic building block. From it, we construct **elementary matrices** — matrices that perform row operations when multiplied on the left.

Definition

An **Elementary Matrix** is a matrix that differs from the identity matrix by a single elementary row operation. It is constructed as:

$$\text{Elementary Matrix} = I + \lambda \times E_{ij}$$

with appropriate choices of i , j , and λ .

3.5.1 Elementary Matrix Construction Using Eij

Row Operation	Elementary Matrix	Formula using Eij
$L_i \leftarrow L_i + \lambda L_j$	$I_n + \lambda E_{i,j}$	$I_n + \lambda E_{i,j}$
$L_i \leftarrow \lambda L_i$	$I_n + (\lambda - 1)E_{i,i}$	$I_n - E_{i,i} + \lambda E_{i,i}$
$L_i \leftrightarrow L_j$	$I_n - E_{i,i} - E_{j,j} + E_{i,j} + E_{j,i}$	$I_n - E_{i,i} - E_{j,j} + E_{i,j} + E_{j,i}$

Table 3.1: Elementary matrix construction using the basis matrix E_{ij} .

Important

- **Eij** is a basis matrix (1 at position (i, j) , 0 elsewhere)
- **Elementary matrices** are built from I and E_{ij}
- Multiplying a matrix by an elementary matrix on the left performs a row operation
- Multiplying a matrix by an elementary matrix on the right performs a column operation

3.5.1 Basis for Arbitrary Matrices

Any matrix A can be constructed as a linear combination of E_{ij} basis matrices:

$$A = \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} a_{ij} E_{ij}$$

This representation is:

- **Unique:** Each matrix has exactly one representation
- **Complete:** Every matrix can be represented
- **Minimal:** No smaller set of basis matrices exists

3.6 Next Steps

In Chapter 4, we will explore the basic matrix operations:

- Matrix addition
- Scalar multiplication
- Matrix multiplication

These operations will be used to combine E_{ij} basis matrices into the elementary matrices required for LU decomposition.

Chapter 4

Operations on Rows as Matrix Products

4.1 Introduction

In traditional Gaussian elimination, row operations are performed directly on the matrix entries using scalar arithmetic. While computationally efficient, this approach obscures the underlying linear algebra.

This chapter presents a fundamentally different approach: **every row operation is represented as a matrix multiplication**. When we multiply a matrix A on the left by an elementary matrix E , the product $E \times A$ performs a specific row operation on A .

Important

The key insight is that row operations are linear transformations, and linear transformations can be represented as matrices. Therefore, any sequence of row operations corresponds to multiplying by a product of elementary matrices.

4.2 Mathematical Foundation

Definition

An **elementary matrix** is a matrix that differs from the identity matrix I_n by a single elementary row operation. When multiplied on the left of another matrix A , it performs that same row operation on A .

Let E be an elementary matrix. Then:

$$E \times A = A_{\text{new}} \quad (4.1)$$

where A_{new} is the result of applying a row operation to A .

The power of this representation becomes evident when multiple operations are applied:

$$E_k \times \cdots \times E_2 \times E_1 \times A = U \quad (4.2)$$

The product $E_k \times \cdots \times E_2 \times E_1$ encodes the entire sequence of row operations.

4.3 Row Scaling: $L_i \leftarrow \lambda L_i$

4.3.1 Mathematical Definition

The row scaling operation multiplies row i by a scalar $\lambda \neq 0$.

Definition

The elementary matrix that scales row i by λ is:

$$E_{\text{scale}}(i, \lambda) = I + (\lambda - 1) \times E_{ii} \quad (4.3)$$

Where I is the identity matrix and E_{ii} is the basis matrix with 1 at position (i, i) .

4.3.2 Example

For a 3×3 matrix, scaling row 1 by $\lambda = 2$:

$$E = I + (2 - 1) \times E_{11} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Multiplying any $3 \times n$ matrix A on the left by this matrix doubles row 1:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix} = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ 2a_{10} & 2a_{11} & 2a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix}$$

4.3.3 Java Implementation

```

1 package com.linsolve.decompositions.lu.components.
   elementaryoperations;
2
3 import com.linsolve.decompositions.lu.components.Identity;
4 import com.linsolve.decompositions.lu.components.Eij;
5 import com.linsolve.decompositions.lu.components.SquareMatrix;
6 import com.linsolve.decompositions.lu.components.basicoperations.*;
7 import com.linsolve.decompositions.lu.components.helper.
   GeneratingRandomMatrix;
8
9 public class Li_lamda_Li extends SquareMatrix {
10     private Integer i;
11     private Integer m;
12     private Eij eii;
13     private Identity id;
14     private Double lamda;
15
16     public Li_lamda_Li(Integer m, Double lamda, Integer i) {

```

```

17     super(m);
18     this.i = i;
19     this.m = m;
20     this.eii = new Eij(i, i, m, m);
21     this.id = new Identity(m);
22     this.lamda = lamda;
23
24     // E = I + (lamda - 1) * Eii
25     // Which is equivalent to: I - Eii + lamda * Eii
26     this.setT(AddingMatrices.add(
27         AddingMatrices.add(
28             MultiplyingMatrixByScalar.multiplyByScalar(this.eii,
29                 -1.0),
30             MultiplyingMatrixByScalar.multiplyByScalar(this.eii,
31                 lamda)
32         ),
33         this.id
34     ).getT());
35 }
36
37 public Li_lamda_Li(Integer m) {
38     super(m);
39     this.m = m;
40     this.n = m;
41 }
42
43 // Getters and setters
44 public Integer getI() { return i; }
45 public void setI(Integer i) { this.i = i; }
46 public Integer getM() { return m; }
47 public void setM(Integer m) { this.m = m; }
48 public Eij getEii() { return eii; }
49 public void setEii(Eij eii) { this.eii = eii; }
50 public Identity getId() { return id; }
51 public void setId(Identity id) { this.id = id; }
52 public Double getLamda() { return lamda; }
53 public void setLamda(Double lamda) { this.lamda = lamda; }
54
55 public static void main(String[] args) {
56     Matrix m0 = GeneratingRandomMatrix.supplyRandomMatrix(5, 5);
57     Li_lamda_Li lilamdali = new Li_lamda_Li(5, 2.0, 0);
58
59     System.out.println("Original matrix:");
60     Matrix.displayMatrix(m0);
61
62     System.out.println("\nAfter scaling row 0 by 2:");
63     Matrix.displayMatrix(MultiplyingMatrices.multiplyMatrix(
64         lilamdali, m0));

```

```

62     }
63 }

```

Listing 4.1: *Li_lamda_Lj.java*

4.4 Row Addition: $L_i \leftarrow L_i + \lambda L_j$

4.4.1 Mathematical Definition

The row addition operation adds λ times row j to row i (with $i \neq j$).

Definition

The elementary matrix that adds λ times row j to row i is:

$$E_{\text{add}}(i, j, \lambda) = I + \lambda \times E_{ij} \quad (4.4)$$

Where I is the identity matrix and E_{ij} is the basis matrix with 1 at position (i, j) .

4.4.2 Example

For a 3×3 matrix, adding $\lambda = 2$ times row 0 to row 2:

$$E = I + 2 \times E_{20} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 2 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 2 & 0 & 1 \end{bmatrix}$$

Multiplying any $3 \times n$ matrix A on the left by this matrix adds $2 \times$ row 0 to row 2:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 2 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix} = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} + 2a_{00} & a_{21} + 2a_{01} & a_{22} + 2a_{02} \end{bmatrix}$$

4.4.3 Java Implementation

```

1 package com.linsolve.decompositions.lu.components.
  elementaryoperations;
2
3 import com.linsolve.decompositions.lu.components.Identity;
4 import com.linsolve.decompositions.lu.components.Eij;
5 import com.linsolve.decompositions.lu.components.SquareMatrix;
6 import com.linsolve.decompositions.lu.components.basicoperations.*;
7
8 public class Li_plus_lamda_Lj extends SquareMatrix {
9     private Integer i;
10    private Integer j;
11    private Integer m;

```

```

12     private Eij eij;
13     private Identity id;
14     private Double lamda;
15
16     public Li_plus_lamda_Lj(Integer m, Double lamda, Integer i,
17         Integer j) {
18         super(m);
19         this.i = i;
20         this.j = j;
21         this.m = m;
22         this.n = m;
23         this.eij = new Eij(i, j, m, m);
24         this.id = new Identity(m);
25         this.lamda = lamda;
26
27         // E = I + lambda * Eij
28         this.setT(AddingMatrices.add(
29             MultiplyingMatrixByScalar.multiplyByScalar(this.eij, this
30                 .lamda),
31             this.id
32         ).getT());
33     }
34
35     public Li_plus_lamda_Lj(Integer n, Integer m) {
36         super(n);
37         this.m = m;
38         this.n = m;
39     }
40
41     // Getters and setters
42     public Integer getI() { return i; }
43     public void setI(Integer i) { this.i = i; }
44     public Integer getJ() { return j; }
45     public void setJ(Integer j) { this.j = j; }
46     public Integer getM() { return m; }
47     public void setM(Integer m) { this.m = m; }
48     public Eij getEij() { return eij; }
49     public void setEij(Eij eij) { this.eij = eij; }
50     public Identity getId() { return id; }
51     public void setId(Identity id) { this.id = id; }
52     public Double getLamda() { return lamda; }
53     public void setLamda(Double lamda) { this.lamda = lamda; }
54 }

```

Listing 4.2: *Li_plus_lamda_Lj.java*

4.5 Row Swapping: $L_i \leftrightarrow L_j$

4.5.1 Mathematical Definition

The row swapping operation exchanges rows i and j .

Definition

The elementary matrix that swaps rows i and j is:

$$E_{\text{swap}}(i, j) = I - E_{ii} - E_{jj} + E_{ij} + E_{ji} \quad (4.5)$$

This matrix is a permutation matrix with ones at positions (i, j) , (j, i) , and (k, k) for $k \neq i, j$.

4.5.2 Example

For a 3×3 matrix, swapping rows 0 and 2:

$$E = I - E_{00} - E_{22} + E_{02} + E_{20} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} - \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

$$E = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

Multiplying any $3 \times n$ matrix A on the left by this matrix swaps rows 0 and 2:

$$\begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix} = \begin{bmatrix} a_{20} & a_{21} & a_{22} \\ a_{10} & a_{11} & a_{12} \\ a_{00} & a_{01} & a_{02} \end{bmatrix}$$

4.5.3 Java Implementation

```

1 package com.linsolve.decompositions.lu.components.
   elementaryoperations;
2
3 import com.linsolve.decompositions.lu.components.Identity;
4 import com.linsolve.decompositions.lu.components.Eij;
5 import com.linsolve.decompositions.lu.components.SquareMatrix;
6 import com.linsolve.decompositions.lu.components.basicoperations.*;
7 import com.linsolve.decompositions.lu.components.helper.
   GeneratingRandomMatrix;
8
9 public class Li_swap_Lj extends SquareMatrix {
10     private Integer i;

```

```

11 private Integer j;
12 private Integer m;
13 private Eij eij;
14 private Eij eji;
15 private Eij eii;
16 private Eij ejj;
17 private Identity id;
18
19 public Li_swap_Lj(Integer m, Integer i, Integer j) {
20     super(m);
21     this.i = i;
22     this.j = j;
23     this.m = m;
24     this.eij = new Eij(i, j, m, m);
25     this.eji = new Eij(j, i, m, m);
26     this.eii = new Eij(i, i, m, m);
27     this.ejj = new Eij(j, j, m, m);
28     this.id = new Identity(m);
29
30     // E = I - Eii - Ejj + Eij + Eji
31     Matrix term1 = AddingMatrices.add(
32         MultiplyingMatrixByScalar.multiplyByScalar(this.eii,
33             -1.0),
34         MultiplyingMatrixByScalar.multiplyByScalar(this.ejj,
35             -1.0)
36     );
37
38     Matrix term2 = AddingMatrices.add(this.eji, this.eij);
39
40     Matrix term3 = AddingMatrices.add(term1, term2);
41
42     this.setT(AddingMatrices.add(this.id, term3).getT());
43 }
44
45 // Getters and setters
46 public Integer getI() { return i; }
47 public void setI(Integer i) { this.i = i; }
48 public Integer getJ() { return j; }
49 public void setJ(Integer j) { this.j = j; }
50 public Integer getM() { return m; }
51 public void setM(Integer m) { this.m = m; }
52 public Eij getEij() { return eij; }
53 public void setEij(Eij eij) { this.eij = eij; }
54 public Eij getEji() { return eji; }
55 public void setEji(Eij eji) { this.eji = eji; }
56 public Eij getEii() { return eii; }
57 public void setEii(Eij eii) { this.eii = eii; }
58 public Eij getEjj() { return ejj; }

```

```

57 public void setEjj(Eij ejj) { this.ejj = ejj; }
58 public Identity getId() { return id; }
59 public void setId(Identity id) { this.id = id; }
60
61 public static void main(String[] args) {
62     Matrix m0 = GeneratingRandomMatrix.supplyRandomMatrix(5, 5);
63     Li_swap_Lj swap = new Li_swap_Lj(5, 2, 3);
64
65     System.out.println("Original matrix:");
66     Matrix.displayMatrix(m0);
67
68     System.out.println("\nAfter swapping rows 2 and 3:");
69     Matrix.displayMatrix(MultiplyingMatrices.multiplyMatrix(swap,
70         m0));
71 }

```

Listing 4.3: *Li_swap_Lj.java*

4.6 Why These Operations are Invertible

Important

Every elementary matrix is invertible, and its inverse is another elementary matrix of the same type:

Operation	Inverse
Scale row i by λ ($\lambda \neq 0$)	Scale row i by $1/\lambda$
Add $\lambda \times$ row j to row i	Add $-\lambda \times$ row j to row i
Swap rows i and j	Swap rows i and j (self-inverse)

This property is crucial for LU decomposition, as it allows us to reconstruct the original matrix from the product of elementary matrices.

4.6.1 Proof of Invertibility

1. **Row scaling:** $E_{\text{scale}}(i, \lambda) \times E_{\text{scale}}(i, 1/\lambda) = I$
2. **Row addition:** $(I + \lambda E_{ij}) \times (I - \lambda E_{ij}) = I$ (since $E_{ij}^2 = 0$ for $i \neq j$)
3. **Row swapping:** $E_{\text{swap}}(i, j) \times E_{\text{swap}}(i, j) = I$ (swapping twice returns to original)

4.7 Summary

This chapter introduced the three fundamental row operations expressed as matrix products:

Operation	Matrix Formula	Java Class	———— ———— ————	$L_i \leftarrow \lambda L_i$
$I + (\lambda - 1)E_{ii}$	Li_lambda_Li	$L_i \leftarrow L_i + \lambda L_j$	$I + \lambda E_{ij}$	Li_plus_lambda_Lj
$I - E_{ii} - E_{jj} + E_{ij} + E_{ji}$	Li_swap_Lj			$L_i \leftrightarrow L_j$

These operations are the building blocks for Gaussian elimination. By composing them as matrix products, we can represent any sequence of row operations as a single matrix.

4.8 Next Steps

In Chapter 6, we will use these elementary matrices to implement Gaussian elimination (forward phase) and accumulate the product of operations to obtain an upper triangular matrix U .

Chapter 5

Forward Elimination (Gauss Pivot)

5.1 Introduction

Forward elimination, also known as Gaussian elimination, is the process of transforming a matrix A into an upper triangular matrix U by applying a sequence of row operations. In this framework, each operation is represented as an elementary matrix, and the entire elimination process is the product of these matrices.

Definition

Forward elimination transforms a square matrix A into an upper triangular matrix U by:

1. Normalizing each pivot row (making the diagonal element equal to 1)
2. Eliminating all elements below each pivot (making them zero)

5.2 The Algorithm

5.2.1 Assumptions

For the initial implementation, we assume:

- The matrix A is square ($n \times n$)
- The matrix is diagonally dominant (no zero pivots)
- No row swapping is required

5.2.2 Step-by-Step Description

For each column j from 0 to $n - 1$:

1. **Normalize the pivot row:** Multiply row j by $1/a_{jj}$ so that $a_{jj} = 1$

2. **Eliminate below the pivot:** For each row i from $j + 1$ to $n - 1$, subtract a_{ij} times row j from row i to make $a_{ij} = 0$

5.2.3 Mathematical Formulation

Let $A^{(0)} = A$. For each pivot column j :

$$\text{Normalization: } A^{(j+1)} = \text{Li_lamda_Li} \left(\frac{1}{a_{jj}^{(j)}} \right) \times A^{(j)} \quad (5.1)$$

$$\text{Elimination: } A^{(j+1)} = \text{Li_plus_lamda_Lj}(-a_{ij}^{(j)}) \times A^{(j)} \quad \text{for each } i > j \quad (5.2)$$

After processing all columns, the final matrix $U = A^{(n)}$ is upper triangular.

5.3 Accumulation of Elementary Matrices

Important

A key insight of this framework is that we accumulate the product of all applied elementary matrices in a separate matrix called `computedMatrix`.

Initially, `computedMatrix` = I (the identity).

After each operation, we update:

$$\text{computedMatrix} = E \times \text{computedMatrix} \quad (5.3)$$

At the end of forward elimination:

$$\text{computedMatrix} = E_k \times \cdots \times E_2 \times E_1 \quad (5.4)$$

where each E is an elementary matrix (either `Li_lamda_Li` or `Li_plus_lamda_Lj`).

5.4 Java Implementation

5.4.1 The ApplyingGaussElimination Class

```

1 package com.linsolve.decompositions.lu.operations;
2
3 import com.linsolve.decompositions.lu.components.Matrix;
4 import com.linsolve.decompositions.lu.components.Identity;
5 import com.linsolve.decompositions.lu.components.basicoperations.
    MultiplyingMatrices;
6 import com.linsolve.decompositions.lu.components.helper.
    GeneratingRandomMatrix;
7 import com.linsolve.decompositions.lu.components.elementaryoperations
    .Li_lamda_Li;

```

```

8 import com.linsolve.decompositions.lu.components.elementaryoperations
   .Li_plus_lamda_Lj;
9
10 public class ApplyingGaussElimination {
11     private Matrix initialMatrix;
12     private Matrix generatedMatrix;
13     private Matrix computedMatrix;
14     private Li_plus_lamda_Lj li_plus_lamda_lj = null;
15     private Li_lamda_Li li_lamda_li = null;
16     private Matrix lower;
17     private int index;
18     private int size;
19
20     public ApplyingGaussElimination(int index, int size) {
21         super();
22         this.index = index;
23         this.size = size;
24         this.initialMatrix = GeneratingRandomMatrix.
25             supplyRandomMatrix(index, size);
26         this.generatedMatrix = new Matrix(this.initialMatrix.getM(),
27             this.initialMatrix.getN());
28         this.generatedMatrix.setT(this.initialMatrix.getT());
29         this.computedMatrix = new Identity(this.initialMatrix.getM())
30             ;
31         this.lower = new Matrix(this.initialMatrix.getM(), this.
32             initialMatrix.getN());
33     }
34
35     public ApplyingGaussElimination(Matrix initialMatrix) {
36         super();
37         this.initialMatrix = initialMatrix;
38         this.generatedMatrix = new Matrix(this.initialMatrix.getM(),
39             this.initialMatrix.getN());
40         this.generatedMatrix.setT(this.initialMatrix.getT());
41         this.computedMatrix = new Identity(this.initialMatrix.getM())
42             ;
43         this.lower = new Matrix(this.initialMatrix.getM(), this.
44             initialMatrix.getM());
45     }
46
47     public void apply_gauss_elimination(int j) {
48         // Step 1: Normalize the pivot row
49         li_lamda_li = new Li_lamda_Li(
50             this.generatedMatrix.getM(),
51             1.0 / this.generatedMatrix.getT()[j][j],
52             j
53         );
54     }

```

```
47     this.generatedMatrix = MultiplyingMatrices.multiplyMatrix(  
48         li_lamda_li, this.generatedMatrix);  
49     this.computedMatrix = MultiplyingMatrices.multiplyMatrix(  
50         li_lamda_li, this.computedMatrix);  
51  
52     // Step 2: Eliminate below the pivot  
53     for (int i = 0; i < generatedMatrix.getM(); i++) {  
54         if (i > j) {  
55             li_plus_lamda_lj = new Li_plus_lamda_Lj(  
56                 generatedMatrix.getM(),  
57                 -generatedMatrix.getT()[i][j],  
58                 i, j  
59             );  
60             generatedMatrix = MultiplyingMatrices.multiplyMatrix(  
61                 li_plus_lamda_lj, generatedMatrix);  
62             computedMatrix = MultiplyingMatrices.multiplyMatrix(  
63                 li_plus_lamda_lj, computedMatrix);  
64         }  
65     }  
66 }  
67  
68 public void apply_gauss_elimination() {  
69     for (int j = 0; j < this.generatedMatrix.getM(); j++) {  
70         this.apply_gauss_elimination(j);  
71     }  
72 }  
73  
74 public void build_lower_matrix() {  
75     this.lower = this.computedMatrix;  
76 }  
77  
78 public void format_matrix_result() {  
79     Double[][] d_array_1 = new Double[this.generatedMatrix.getM()  
80         ][this.generatedMatrix.getM()];  
81     for (int i = 0; i < this.generatedMatrix.getM(); i++) {  
82         for (int j = 0; j < this.generatedMatrix.getM(); j++) {  
83             if (i > j) {  
84                 d_array_1[i][j] = 0.0;  
85             } else if (i < j) {  
86                 d_array_1[i][j] = this.generatedMatrix.getT()[i][  
87                     j];  
88             } else {  
89                 d_array_1[i][j] = 1.0;  
90             }  
91         }  
92     }  
93     this.generatedMatrix.setT(d_array_1);  
94 }
```

```

89
90     public void apply() {
91         this.apply_gauss_elimination();
92         this.format_matrix_result();
93         this.build_lower_matrix();
94     }
95
96     // Getters and setters
97     public Matrix getInitialMatrix() { return initialMatrix; }
98     public void setInitialMatrix(Matrix initialMatrix) { this.
99         initialMatrix = initialMatrix; }
100    public Matrix getGeneratedMatrix() { return generatedMatrix; }
101    public void setGeneratedMatrix(Matrix generatedMatrix) { this.
102        generatedMatrix = generatedMatrix; }
103    public Matrix getComputedMatrix() { return computedMatrix; }
104    public void setComputedMatrix(Matrix computedMatrix) { this.
105        computedMatrix = computedMatrix; }
106    public Li_plus_lamda_Lj getLi_plus_lamda_lj() { return
107        li_plus_lamda_lj; }
108    public void setLi_plus_lamda_lj(Li_plus_lamda_Lj li_plus_lamda_lj
109        ) { this.li_plus_lamda_lj = li_plus_lamda_lj; }
110    public Li_lamda_Li getLi_lamda_li() { return li_lamda_li; }
111    public void setLi_lamda_li(Li_lamda_Li li_lamda_li) { this.
112        li_lamda_li = li_lamda_li; }
113    public Matrix getLower() { return lower; }
114    public void setLower(Matrix lower) { this.lower = lower; }
115    public int getIndex() { return index; }
116    public void setIndex(int index) { this.index = index; }
117    public int getSize() { return size; }
118    public void setSize(int size) { this.size = size; }
119 }

```

Listing 5.1: ApplyingGaussElimination.java

5.5 Step-by-Step Example

Consider the 3×3 matrix:

$$A = \begin{bmatrix} 4 & 1 & 1 \\ 2 & 5 & 1 \\ 1 & 2 & 6 \end{bmatrix}$$

5.5.1 Step 1: Column 0 ($j = 0$)

Normalize row 0: $\lambda = 1/4 = 0.25$

$$E_1 = \text{Li_lamda_Li}(0, 0.25) = \begin{bmatrix} 0.25 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$A^{(1)} = E_1 \times A = \begin{bmatrix} 1 & 0.25 & 0.25 \\ 2 & 5 & 1 \\ 1 & 2 & 6 \end{bmatrix}$$

Eliminate row 1: $\lambda = -2$ (add $-2 \times$ row 0 to row 1)

$$E_2 = \text{Li_plus_lamda_Lj}(1, 0, -2) = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$A^{(2)} = E_2 \times A^{(1)} = \begin{bmatrix} 1 & 0.25 & 0.25 \\ 0 & 4.5 & 0.5 \\ 1 & 2 & 6 \end{bmatrix}$$

Eliminate row 2: $\lambda = -1$

$$E_3 = \text{Li_plus_lamda_Lj}(2, 0, -1) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix}$$

$$A^{(3)} = E_3 \times A^{(2)} = \begin{bmatrix} 1 & 0.25 & 0.25 \\ 0 & 4.5 & 0.5 \\ 0 & 1.75 & 5.75 \end{bmatrix}$$

5.5.2 Step 2: Column 1 ($j = 1$)

Normalize row 1: $\lambda = 1/4.5 \approx 0.2222$

$$E_4 = \text{Li_lamda_Li}(1, 0.2222) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.2222 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$A^{(4)} = E_4 \times A^{(3)} = \begin{bmatrix} 1 & 0.25 & 0.25 \\ 0 & 1 & 0.1111 \\ 0 & 1.75 & 5.75 \end{bmatrix}$$

Eliminate row 2: $\lambda = -1.75$

$$E_5 = \text{Li_plus_lamda_Lj}(2, 1, -1.75) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -1.75 & 1 \end{bmatrix}$$

$$A^{(5)} = E_5 \times A^{(4)} = \begin{bmatrix} 1 & 0.25 & 0.25 \\ 0 & 1 & 0.1111 \\ 0 & 0 & 5.5556 \end{bmatrix}$$

5.5.3 Step 3: Column 2 ($j = 2$)

Normalize row 2: $\lambda = 1/5.5556 \approx 0.18$

$$E_6 = \text{Li_lamda_Li}(2, 0.18) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0.18 \end{bmatrix}$$

$$U = E_6 \times A^{(5)} = \begin{bmatrix} 1 & 0.25 & 0.25 \\ 0 & 1 & 0.1111 \\ 0 & 0 & 1 \end{bmatrix}$$

5.5.4 Result

The final upper triangular matrix U :

$$U = \begin{bmatrix} 1 & 0.25 & 0.25 \\ 0 & 1 & 0.1111 \\ 0 & 0 & 1 \end{bmatrix}$$

The accumulated product of elementary matrices $E_{\text{total}} = E_6 \times E_5 \times E_4 \times E_3 \times E_2 \times E_1$ is stored in `computedMatrix`.

5.6 Formatting the Result

After forward elimination, the `format_matrix_result()` method transforms the matrix into a clean upper triangular form with ones on the diagonal:

```

1 public void format_matrix_result() {
2     Double[][] d_array_1 = new Double[this.generatedMatrix.getM()][
3         this.generatedMatrix.getM()];
4     for (int i = 0; i < this.generatedMatrix.getM(); i++) {
5         for (int j = 0; j < this.generatedMatrix.getM(); j++) {
6             if (i > j) {
7                 d_array_1[i][j] = 0.0;           // Below diagonal:
8                 zero
9             } else if (i < j) {
10                d_array_1[i][j] = this.generatedMatrix.getT()[i][j];
11                // Above diagonal: keep
12            } else {
13                d_array_1[i][j] = 1.0;           // Diagonal: one
14            }
15        }
16    }
17    this.generatedMatrix.setT(d_array_1);
18 }

```

Listing 5.2: Formatting the result

5.7 Summary

This chapter presented the forward elimination phase of LU decomposition:

Phase	Operations	Result	Normalization	Li_lamda_Li
Pivot = 1	Elimination	Li_plus_lamda_Lj	Zeros below pivot	Accumulation
$E \times \text{computedMatrix}$	Product of all operations	Formatting	Set zeros and ones	Clean upper triangular U

After forward elimination:

- `generatedMatrix` contains the upper triangular matrix U
- `computedMatrix` contains the product $E_{\text{total}} = E_k \times \cdots \times E_1$

5.8 Next Steps

In Chapter 7, we will perform the backward elimination phase (back substitution) to transform U into the identity matrix and complete the computation of the inverse.

Chapter 6

Backward Elimination (Back Substitution)

6.1 Introduction

After forward elimination, we have an upper triangular matrix U and an accumulated product E_{forward} of all elementary matrices used. However, forward elimination alone does not give us the inverse of A or the complete LU decomposition.

The backward elimination phase (also called back substitution) continues the process by eliminating elements **above** the diagonal, transforming U into the identity matrix I .

Definition

Backward elimination transforms an upper triangular matrix U into the identity matrix I by:

1. Working from the last column backward to the first
2. Eliminating all elements above each pivot (making them zero)

6.2 The Algorithm

6.2.1 Mathematical Formulation

Starting from the upper triangular matrix U obtained after forward elimination:

For each column j from $n - 1$ down to 1:

- For each row i from 0 to $j - 1$:

$$L_{i_plus_lamda_Lj}(i, j, -u_{ij}) \times U \tag{6.1}$$

This eliminates the element at position (i, j) by subtracting u_{ij} times row j from row i .

After processing all columns, the resulting matrix becomes the identity I .

6.2.2 Accumulation

Similar to forward elimination, we accumulate the product of all backward elimination matrices in a separate matrix called `finalComputedMatrix`:

$$\text{finalComputedMatrix} = F_m \times \cdots \times F_2 \times F_1 \quad (6.2)$$

At the end of backward elimination:

$$\text{finalComputedMatrix} \times U = I \quad (6.3)$$

Therefore:

$$\text{finalComputedMatrix} = U^{-1} \quad (6.4)$$

6.3 The Complete Inverse

Important

Let:

- E_{forward} = product of all forward elimination matrices
- E_{backward} = product of all backward elimination matrices

Then:

$$E_{\text{backward}} \times E_{\text{forward}} \times A = I \quad (6.5)$$

Therefore:

$$\boxed{A^{-1} = E_{\text{backward}} \times E_{\text{forward}}} \quad (6.6)$$

The inverse of A is simply the product of all elementary matrices from both phases!

6.4 Java Implementation

6.4.1 The ApplyingBackSubstitution Class

```

1 package com.linsolve.decompositions.lu.operations;
2
3 import com.linsolve.decompositions.lu.components.Matrix;
4 import com.linsolve.decompositions.lu.components.Identity;
5 import com.linsolve.decompositions.lu.components.basicoperations.*;
6 import com.linsolve.decompositions.lu.components.helper.*;
7 import com.linsolve.decompositions.lu.components.elementaryoperations
   .*;
8
9 public class ApplyingBackSubstitution {
10

```

```

11 private ApplyingGaussElimination operator;
12 private Matrix initialMatrix;
13 private Matrix generatedMatrix;
14 private Matrix computedMatrix;
15 private Matrix finalGeneratedMatrix;
16 private Matrix finalComputedMatrix;
17 private Li_plus_lamda_Lj li_plus_lamda_lj = null;
18 private Li_lamda_Li li_lamda_li = null;
19 private Matrix upper;
20 private Matrix lower;
21 private Matrix inverse;
22 private Matrix l;
23 private Matrix u;
24 private Matrix reconstructed;
25 private InvertingLowerMatrix ilm;
26 private InvertingUpperMatrix ium;
27 private int index;
28 private int size;
29
30 public ApplyingBackSubstitution(int index, int size) {
31     super();
32     this.index = index;
33     this.size = size;
34     this.operator = new ApplyingGaussElimination(index, size);
35     this.initialMatrix = this.operator.getInitialMatrix();
36     this.operator.apply_gauss_elimination();
37     this.operator.format_matrix_result();
38     this.operator.build_lower_matrix();
39     this.generatedMatrix = operator.getGeneratedMatrix();
40     this.computedMatrix = new Identity(this.initialMatrix.getM())
41     ;
42     this.computedMatrix = operator.getComputedMatrix();
43     this.finalGeneratedMatrix = new Matrix(this.initialMatrix.
44         getM(), this.initialMatrix.getN());
45     this.finalGeneratedMatrix.setT(this.generatedMatrix.getT());
46     this.finalComputedMatrix = new Identity(operator.
47         getInitialMatrix().getM());
48     this.li_lamda_li = new Li_lamda_Li(this.operator.
49         getInitialMatrix().getM());
50     this.li_plus_lamda_lj = new Li_plus_lamda_Lj(
51         this.operator.getInitialMatrix().getM(),
52         this.operator.getInitialMatrix().getM()
53     );
54     this.lower = new Matrix(this.initialMatrix.getM(), this.
55         getInitialMatrix().getN());
56     this.lower = this.operator.getLower();
57     this.upper = new Matrix(this.initialMatrix.getM(), this.
58         getInitialMatrix().getN());

```

```

53     this.l = new Matrix(this.initialMatrix.getM(), this.
        getInitialMatrix().getN());
54     this.u = new Matrix(this.initialMatrix.getM(), this.
        getInitialMatrix().getN());
55     this.inverse = new Matrix(this.initialMatrix.getM(), this.
        getInitialMatrix().getN());
56     this.reconstructed = new Matrix(this.initialMatrix.getM(),
        this.getInitialMatrix().getN());
57 }
58
59 public ApplyingBackSubstitution(ApplyingGaussElimination operator
    ) {
60     this.operator = operator;
61     this.initialMatrix = this.operator.getInitialMatrix();
62     this.operator.apply();
63     this.generatedMatrix = this.operator.getGeneratedMatrix();
64     this.computedMatrix = new Identity(this.initialMatrix.getM())
        ;
65     this.computedMatrix = operator.getComputedMatrix();
66     this.finalGeneratedMatrix = new Matrix(this.initialMatrix.
        getM(), this.initialMatrix.getN());
67     this.finalGeneratedMatrix.setT(this.generatedMatrix.getT());
68     this.finalComputedMatrix = new Identity(operator.
        getInitialMatrix().getM());
69     this.li_lamda_li = new Li_lamda_Li(this.operator.
        getInitialMatrix().getM());
70     this.li_plus_lamda_lj = new Li_plus_lamda_Lj(
71         this.operator.getInitialMatrix().getM(),
72         this.operator.getInitialMatrix().getM()
73     );
74     this.lower = new Matrix(this.initialMatrix.getM(), this.
        getInitialMatrix().getN());
75     this.lower = this.operator.getLower();
76     this.upper = new Matrix(this.initialMatrix.getM(), this.
        getInitialMatrix().getN());
77     this.l = new Matrix(this.initialMatrix.getM(), this.
        getInitialMatrix().getN());
78     this.u = new Matrix(this.initialMatrix.getM(), this.
        getInitialMatrix().getN());
79     this.inverse = new Matrix(this.initialMatrix.getM(), this.
        getInitialMatrix().getN());
80 }
81
82 public void apply_back_substitution(int j) {
83     li_plus_lamda_lj = null;
84     for (int i = 0; i < this.finalGeneratedMatrix.getM(); i++) {
85         if (i < j) {
86             li_plus_lamda_lj = new Li_plus_lamda_Lj(

```

```
87         this.finalGeneratedMatrix.getM(),
88         -this.finalGeneratedMatrix.getT()[i][j],
89         i, j
90     );
91     this.finalGeneratedMatrix = MultiplyingMatrices.
        multiplyMatrix(
92         li_plus_lamda_lj,
93         this.finalGeneratedMatrix
94     );
95     this.finalComputedMatrix = MultiplyingMatrices.
        multiplyMatrix(
96         li_plus_lamda_lj,
97         this.finalComputedMatrix
98     );
99     }
100 }
101 }
102
103 public void apply_back_substitution() {
104     for (int i = finalGeneratedMatrix.getM() - 1; i > 0; i--) {
105         this.apply_back_substitution(i);
106     }
107 }
108
109 public void build_upper_matrix() {
110     this.upper = this.getFinalComputedMatrix();
111 }
112
113 public void build_inverse_matrix() {
114     this.inverse = MultiplyingMatrices.multiplyMatrix(this.upper,
        this.lower);
115 }
116
117 public void build_reconstructed_matrix() {
118     this.ilm = new InvertingLowerMatrix(this.lower);
119     this.ilm.apply();
120     this.l = this.ilm.getInverse();
121     this.ium = new InvertingUpperMatrix(this.upper);
122     this.ium.apply();
123     this.u = this.ium.getInverse();
124 }
125
126 public void apply() {
127     this.apply_back_substitution();
128     this.build_upper_matrix();
129     this.build_inverse_matrix();
130     this.build_reconstructed_matrix();
131 }
```

```

132
133 // Getters and setters
134 public ApplyingGaussElimination getOperator() { return operator;
    }
135 public void setOperator(ApplyingGaussElimination operator) { this
    .operator = operator; }
136 public Matrix getInitialMatrix() { return initialMatrix; }
137 public void setInitialMatrix(Matrix initialMatrix) { this.
    initialMatrix = initialMatrix; }
138 public Matrix getGeneratedMatrix() { return generatedMatrix; }
139 public void setGeneratedMatrix(Matrix generatedMatrix) { this.
    generatedMatrix = generatedMatrix; }
140 public Matrix getComputedMatrix() { return computedMatrix; }
141 public void setComputedMatrix(Matrix computedMatrix) { this.
    computedMatrix = computedMatrix; }
142 public Matrix getFinalGeneratedMatrix() { return
    finalGeneratedMatrix; }
143 public void setFinalGeneratedMatrix(Matrix finalGeneratedMatrix)
    { this.finalGeneratedMatrix = finalGeneratedMatrix; }
144 public Matrix getFinalComputedMatrix() { return
    finalComputedMatrix; }
145 public void setFinalComputedMatrix(Matrix finalComputedMatrix) {
    this.finalComputedMatrix = finalComputedMatrix; }
146 public Li_plus_lamda_Lj getLi_plus_lamda_lj() { return
    li_plus_lamda_lj; }
147 public void setLi_plus_lamda_lj(Li_plus_lamda_Lj li_plus_lamda_lj
    ) { this.li_plus_lamda_lj = li_plus_lamda_lj; }
148 public Li_lamda_Li getLi_lamda_li() { return li_lamda_li; }
149 public void setLi_lamda_li(Li_lamda_Li li_lamda_li) { this.
    li_lamda_li = li_lamda_li; }
150 public Matrix getUpper() { return upper; }
151 public void setUpper(Matrix upper) { this.upper = upper; }
152 public Matrix getLower() { return lower; }
153 public void setLower(Matrix lower) { this.lower = lower; }
154 public Matrix getInverse() { return inverse; }
155 public void setInverse(Matrix inverse) { this.inverse = inverse;
    }
156 public Matrix getL() { return l; }
157 public void setL(Matrix l) { this.l = l; }
158 public Matrix getU() { return u; }
159 public void setU(Matrix u) { this.u = u; }
160 public Matrix getReconstructed() { return reconstructed; }
161 public void setReconstructed(Matrix reconstructed) { this.
    reconstructed = reconstructed; }
162 public InvertingLowerMatrix getIlm() { return ilm; }
163 public void setIlm(InvertingLowerMatrix ilm) { this.ilm = ilm; }
164 public InvertingUpperMatrix getIum() { return ium; }
165 public void setIum(InvertingUpperMatrix ium) { this.ium = ium; }

```



```

166 public int getIndex() { return index; }
167 public void setIndex(int index) { this.index = index; }
168 public int getSize() { return size; }
169 public void setSize(int size) { this.size = size; }
170 }

```

Listing 6.1: ApplyingBackSubstitution.java

6.5 Step-by-Step Example (Continued)

Continuing from Chapter 6, we had:

$$U = \begin{bmatrix} 1 & 0.25 & 0.25 \\ 0 & 1 & 0.1111 \\ 0 & 0 & 1 \end{bmatrix}$$

6.5.1 Step 1: Column 2 ($j = 2$)

We eliminate above the diagonal in column 2.

Eliminate row 1: $\lambda = -0.1111$ (subtract $0.1111 \times$ row 2 from row 1)

$$F_1 = \text{Li_plus_lamda_Lj}(1, 2, -0.1111) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & -0.1111 \\ 0 & 0 & 1 \end{bmatrix}$$

$$U^{(1)} = F_1 \times U = \begin{bmatrix} 1 & 0.25 & 0.25 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Eliminate row 0: $\lambda = -0.25$

$$F_2 = \text{Li_plus_lamda_Lj}(0, 2, -0.25) = \begin{bmatrix} 1 & 0 & -0.25 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$U^{(2)} = F_2 \times U^{(1)} = \begin{bmatrix} 1 & 0.25 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

6.5.2 Step 2: Column 1 ($j = 1$)

Eliminate row 0: $\lambda = -0.25$

$$F_3 = \text{Li_plus_lamda_Lj}(0, 1, -0.25) = \begin{bmatrix} 1 & -0.25 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$I = F_3 \times U^{(2)} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

6.5.3 Result

After backward elimination, the matrix becomes the identity I , and:

$$\text{finalComputedMatrix} = F_3 \times F_2 \times F_1 = U^{-1} \quad (6.7)$$

$$A^{-1} = \text{finalComputedMatrix} \times \text{computedMatrix} \quad (6.8)$$

6.6 Why This Works

Theorem

Let E_{forward} be the product of all forward elimination matrices, and E_{backward} be the product of all backward elimination matrices. Then:

$$E_{\text{backward}} \times E_{\text{forward}} \times A = I \quad (6.9)$$

Therefore:

$$A^{-1} = E_{\text{backward}} \times E_{\text{forward}} \quad (6.10)$$

6.6.1 Proof

1. By construction, $E_{\text{forward}} \times A = U$ (upper triangular)
2. By construction, $E_{\text{backward}} \times U = I$
3. Substituting: $E_{\text{backward}} \times (E_{\text{forward}} \times A) = I$
4. By associativity: $(E_{\text{backward}} \times E_{\text{forward}}) \times A = I$
5. Therefore $A^{-1} = E_{\text{backward}} \times E_{\text{forward}}$

6.7 Summary

This chapter presented the backward elimination phase:

Phase	Operation	Result					Backward elimination	Li_plus_lambda_Lj
with $i < j$	Zeros above diagonal	Accumulation	$F \times$	finalComputedMatrix	Product of	backward operations	Final result	$E_{\text{backward}} \times E_{\text{forward}} $
							$A^{-1} $	

After both forward and backward elimination:

- `computedMatrix` = E_{forward}

- `finalComputedMatrix` = $E_{\text{backward}} = U^{-1}$
- The product `finalComputedMatrix` $\times$ `computedMatrix` = A^{-1}

6.8 Next Steps

In Chapter 8, we will extract the L and U matrices from the accumulated products and complete the LU decomposition.

Chapter 7

Extracting L and U via Triangular Inversion

7.1 Introduction

After completing forward and backward elimination, we have computed the inverse of the original matrix:

$$A^{-1} = E_{\text{backward}} \times E_{\text{forward}} \quad (7.1)$$

However, our goal is the LU decomposition: $A = L \times U$. This chapter shows how to extract L and U from the accumulated products using triangular matrix inversion.

7.2 The Relationship Between A^{-1} , L, and U

Theorem

If $A = L \times U$, then:

$$A^{-1} = U^{-1} \times L^{-1} \quad (7.2)$$

This is a direct consequence of the matrix inversion property: $(LU)^{-1} = U^{-1}L^{-1}$.

Important

From forward elimination, we have:

- `computedMatrix` = E_{forward}
- $E_{\text{forward}} \times A = U$

From backward elimination, we have:

- `finalComputedMatrix` = $E_{\text{backward}} = U^{-1}$

Therefore:

$$A^{-1} = U^{-1} \times L^{-1} \quad (7.3)$$

But we also have $A^{-1} = E_{\text{backward}} \times E_{\text{forward}} = U^{-1} \times E_{\text{forward}}$

This implies:

$$L^{-1} = E_{\text{forward}} \quad (7.4)$$

7.3 Extracting L and U

7.3.1 Key Insight

From the relationships above:

$$L = (E_{\text{forward}})^{-1} \quad (7.5)$$

$$U = (E_{\text{backward}})^{-1} \quad (7.6)$$

Where:

- E_{forward} is lower triangular (product of row operations below the diagonal)
- E_{backward} is upper triangular (product of row operations above the diagonal)

7.3.2 Why Triangular Matrices Are Easy to Invert

Triangular matrices have special properties that make inversion straightforward:

1. The inverse of a lower triangular matrix is also lower triangular
2. The inverse of an upper triangular matrix is also upper triangular
3. Inversion can be performed in $O(n^2)$ operations using forward/backward substitution

7.4 Inverting Lower Triangular Matrices

7.4.1 Algorithm

For a lower triangular matrix L with ones on the diagonal, its inverse L^{-1} can be computed using forward substitution:

$$(L \times L^{-1})_{ij} = \sum_{k=0}^{n-1} l_{ik} \cdot (L^{-1})_{kj} = \delta_{ij} \quad (7.7)$$

Since $l_{ii} = 1$, we can solve for $(L^{-1})_{ij}$ column by column.

7.4.2 Java Implementation

```

1 package com.linsolve.decompositions.lu.components.helper;
2
3 import com.linsolve.decompositions.lu.components.Matrix;
4 import com.linsolve.decompositions.lu.components.Identity;
5 import com.linsolve.decompositions.lu.components.basicoperations.*;
6 import com.linsolve.decompositions.lu.components.elementaryoperations
   .*;
7
8 public class InvertingLowerMatrix {
9     private Matrix initialMatrix;
10    private Matrix generatedMatrix;
11    private Matrix computedMatrix;
12    private Li_plus_lamda_Lj li_plus_lamda_lj;
13    private Li_lamda_Li li_lamda_li;
14    private Matrix inverse;
15    private Matrix product;
16    private int index;
17    private int size;
18
19    public InvertingLowerMatrix(int index, int size) {
20        super();
21        this.index = index;
22        this.size = size;
23        this.initialMatrix = new Matrix(size, size);
24        this.initialMatrix.setT(GeneratingRandomLowerMatrix.
25            supplyRandomLowerMatrix(index, size).getT());
26        this.generatedMatrix = new Matrix(this.initialMatrix.getM(),
27            this.initialMatrix.getN());
28        this.generatedMatrix.setT(this.initialMatrix.getT());
29        this.computedMatrix = new Identity(this.initialMatrix.getM())
30            ;
31        this.li_lamda_li = new Li_lamda_Li(size);
32        this.li_plus_lamda_lj = new Li_plus_lamda_Lj(size, size);

```

```

30         this.inverse = new Matrix(this.initialMatrix.getM(), this.
           getInitialMatrix().getN());
31         this.product = new Matrix(this.initialMatrix.getM(), this.
           getInitialMatrix().getN());
32     }
33
34     public InvertingLowerMatrix(Matrix initialMatrix) {
35         super();
36         this.initialMatrix = initialMatrix;
37         this.size = this.initialMatrix.getM();
38         this.generatedMatrix = new Matrix(this.initialMatrix.getM(),
           this.initialMatrix.getN());
39         this.generatedMatrix.setT(this.initialMatrix.getT());
40         this.computedMatrix = new Identity(this.initialMatrix.getM())
           ;
41         this.li_lamda_li = new Li_lamda_Li(this.initialMatrix.getM())
           ;
42         this.li_plus_lamda_lj = new Li_plus_lamda_Lj(this.
           initialMatrix.getM(), this.initialMatrix.getM());
43         this.inverse = new Matrix(this.initialMatrix.getM(), this.
           getInitialMatrix().getN());
44         this.product = new Matrix(this.initialMatrix.getM(), this.
           getInitialMatrix().getN());
45     }
46
47     public void apply_gauss_elimination(int j) {
48         li_lamda_li = new Li_lamda_Li(
49             this.generatedMatrix.getM(),
50             1.0 / this.generatedMatrix.getT()[j][j],
51             j
52         );
53         this.generatedMatrix = MultiplyingMatrices.multiplyMatrix(
           li_lamda_li, this.generatedMatrix);
54         this.computedMatrix = MultiplyingMatrices.multiplyMatrix(
           li_lamda_li, this.computedMatrix);
55     }
56
57     public void apply_gauss_elimination() {
58         for (int j = 0; j < this.generatedMatrix.getM(); j++) {
59             this.apply_gauss_elimination(j);
60         }
61     }
62
63     public void apply_back_substitution(int j) {
64         li_plus_lamda_lj = null;
65         for (int i = 0; i < this.generatedMatrix.getM(); i++) {
66             if (i > j) {
67                 li_plus_lamda_lj = new Li_plus_lamda_Lj(

```



```

68         this.generatedMatrix.getM(),
69         -this.generatedMatrix.getT()[i][j],
70         i, j
71     );
72     this.generatedMatrix = MultiplyingMatrices.
        multiplyMatrix(li_plus_lamda_lj, this.
        generatedMatrix);
73     this.computedMatrix = MultiplyingMatrices.
        multiplyMatrix(li_plus_lamda_lj, this.
        computedMatrix);
74     }
75 }
76 }
77
78 public void apply_back_substitution() {
79     for (int j = generatedMatrix.getM() - 1; j >= 0; j--) {
80         this.apply_back_substitution(j);
81     }
82 }
83
84 public void build_upper_matrix() {
85     this.inverse = this.getComputedMatrix();
86 }
87
88 public void build_inverse_matrix() {
89     this.product = MultiplyingMatrices.multiplyMatrix(this.
        inverse, this.initialMatrix);
90 }
91
92 public void apply() {
93     this.apply_gauss_elimination();
94     this.apply_back_substitution();
95     this.build_upper_matrix();
96     this.build_inverse_matrix();
97 }
98
99 // Getters and setters
100 public Matrix getInitialMatrix() { return initialMatrix; }
101 public void setInitialMatrix(Matrix initialMatrix) { this.
    initialMatrix = initialMatrix; }
102 public Matrix getGeneratedMatrix() { return generatedMatrix; }
103 public void setGeneratedMatrix(Matrix generatedMatrix) { this.
    generatedMatrix = generatedMatrix; }
104 public Matrix getComputedMatrix() { return computedMatrix; }
105 public void setComputedMatrix(Matrix computedMatrix) { this.
    computedMatrix = computedMatrix; }
106 public Li_plus_lamda_Lj getLi_plus_lamda_lj() { return
    li_plus_lamda_lj; }

```

```

107 public void setLi_plus_lamda_lj(Li_plus_lamda_Lj li_plus_lamda_lj
    ) { this.li_plus_lamda_lj = li_plus_lamda_lj; }
108 public Li_lamda_Li getLi_lamda_li() { return li_lamda_li; }
109 public void setLi_lamda_li(Li_lamda_Li li_lamda_li) { this.
    li_lamda_li = li_lamda_li; }
110 public Matrix getInverse() { return inverse; }
111 public void setInverse(Matrix inverse) { this.inverse = inverse;
    }
112 public Matrix getProduct() { return product; }
113 public void setProduct(Matrix product) { this.product = product;
    }
114 public int getIndex() { return index; }
115 public void setIndex(int index) { this.index = index; }
116 public int getSize() { return size; }
117 public void setSize(int size) { this.size = size; }
118 }

```

Listing 7.1: InvertingLowerMatrix.java

7.5 Inverting Upper Triangular Matrices

7.5.1 Algorithm

For an upper triangular matrix U , its inverse U^{-1} can be computed using backward substitution:

$$(U \times U^{-1})_{ij} = \sum_{k=0}^{n-1} u_{ik} \cdot (U^{-1})_{kj} = \delta_{ij} \quad (7.8)$$

Since $u_{ii} \neq 0$, we can solve for $(U^{-1})_{ij}$ row by row starting from the bottom.

7.5.2 Java Implementation

```

1 package com.linsolve.decompositions.lu.components.helper;
2
3 import com.linsolve.decompositions.lu.components.Matrix;
4 import com.linsolve.decompositions.lu.components.Identity;
5 import com.linsolve.decompositions.lu.components.basicoperations.*;
6 import com.linsolve.decompositions.lu.components.elementaryoperations
    .*;
7
8 public class InvertingUpperMatrix {
9     private Matrix initialMatrix;
10    private Matrix generatedMatrix;
11    private Matrix computedMatrix;
12    private Li_plus_lamda_Lj li_plus_lamda_lj;

```

```

13 private Li_lamda_Li li_lamda_li;
14 private Matrix inverse;
15 private Matrix product;
16 private int index;
17 private int size;
18
19 public InvertingUpperMatrix(int index, int size) {
20     super();
21     this.index = index;
22     this.size = size;
23     this.initialMatrix = new Matrix(size, size);
24     this.initialMatrix.setT(GeneratingRandomUpperMatrix.
25         supplyRandomUpperMatrix(index, size).getT());
26     this.generatedMatrix = new Matrix(this.initialMatrix.getM(),
27         this.initialMatrix.getN());
28     this.generatedMatrix.setT(this.initialMatrix.getT());
29     this.computedMatrix = new Identity(this.initialMatrix.getM())
30         ;
31     this.li_lamda_li = new Li_lamda_Li(size);
32     this.li_plus_lamda_lj = new Li_plus_lamda_Lj(size, size);
33     this.inverse = new Matrix(this.initialMatrix.getM(), this.
34         getInitialMatrix().getN());
35     this.product = new Matrix(this.initialMatrix.getM(), this.
36         getInitialMatrix().getN());
37 }
38
39 public InvertingUpperMatrix(Matrix initialMatrix) {
40     super();
41     this.initialMatrix = initialMatrix;
42     this.size = this.initialMatrix.getM();
43     this.generatedMatrix = new Matrix(this.initialMatrix.getM(),
44         this.initialMatrix.getN());
45     this.generatedMatrix.setT(this.initialMatrix.getT());
46     this.computedMatrix = new Identity(this.initialMatrix.getM())
47         ;
48     this.li_lamda_li = new Li_lamda_Li(size);
49     this.li_plus_lamda_lj = new Li_plus_lamda_Lj(size, size);
50     this.inverse = new Matrix(this.initialMatrix.getM(), this.
51         getInitialMatrix().getN());
52     this.product = new Matrix(this.initialMatrix.getM(), this.
53         getInitialMatrix().getN());
54 }
55
56 public void apply_gauss_elimination(int j) {
57     li_lamda_li = new Li_lamda_Li(
58         this.generatedMatrix.getM(),
59         1.0 / this.generatedMatrix.getT()[j][j],
60         j

```

```
52     );
53     this.generatedMatrix = MultiplyingMatrices.multiplyMatrix(
54         li_lamda_li, this.generatedMatrix);
55     this.computedMatrix = MultiplyingMatrices.multiplyMatrix(
56         li_lamda_li, this.computedMatrix);
57 }
58
59 public void apply_gauss_elimination() {
60     for (int j = 0; j < this.generatedMatrix.getM(); j++) {
61         this.apply_gauss_elimination(j);
62     }
63 }
64
65 public void apply_back_substitution(int j) {
66     li_plus_lamda_lj = null;
67     for (int i = 0; i < this.generatedMatrix.getM(); i++) {
68         if (i < j) {
69             li_plus_lamda_lj = new Li_plus_lamda_Lj(
70                 this.generatedMatrix.getM(),
71                 -this.generatedMatrix.getT()[i][j],
72                 i, j
73             );
74             this.generatedMatrix = MultiplyingMatrices.
75                 multiplyMatrix(li_plus_lamda_lj, this.
76                     generatedMatrix);
77             this.computedMatrix = MultiplyingMatrices.
78                 multiplyMatrix(li_plus_lamda_lj, this.
79                     computedMatrix);
80         }
81     }
82 }
83
84 public void apply_back_substitution() {
85     for (int j = generatedMatrix.getM() - 1; j >= 0; j--) {
86         this.apply_back_substitution(j);
87     }
88 }
89
90 public void build_upper_matrix() {
91     this.inverse = this.getComputedMatrix();
92 }
93
94 public void build_inverse_matrix() {
95     this.product = MultiplyingMatrices.multiplyMatrix(this.
96         inverse, this.initialMatrix);
97 }
98
99 public void apply() {
```

```

93         this.apply_gauss_elimination();
94         this.apply_back_substitution();
95         this.build_upper_matrix();
96         this.build_inverse_matrix();
97     }
98
99     // Getters and setters
100     public Matrix getInitialMatrix() { return initialMatrix; }
101     public void setInitialMatrix(Matrix initialMatrix) { this.
        initialMatrix = initialMatrix; }
102     public Matrix getGeneratedMatrix() { return generatedMatrix; }
103     public void setGeneratedMatrix(Matrix generatedMatrix) { this.
        generatedMatrix = generatedMatrix; }
104     public Matrix getComputedMatrix() { return computedMatrix; }
105     public void setComputedMatrix(Matrix computedMatrix) { this.
        computedMatrix = computedMatrix; }
106     public Li_plus_lamda_Lj getLi_plus_lamda_lj() { return
        li_plus_lamda_lj; }
107     public void setLi_plus_lamda_lj(Li_plus_lamda_Lj li_plus_lamda_lj
        ) { this.li_plus_lamda_lj = li_plus_lamda_lj; }
108     public Li_lamda_Li getLi_lamda_li() { return li_lamda_li; }
109     public void setLi_lamda_li(Li_lamda_Li li_lamda_li) { this.
        li_lamda_li = li_lamda_li; }
110     public Matrix getInverse() { return inverse; }
111     public void setInverse(Matrix inverse) { this.inverse = inverse;
        }
112     public Matrix getProduct() { return product; }
113     public void setProduct(Matrix product) { this.product = product;
        }
114     public int getIndex() { return index; }
115     public void setIndex(int index) { this.index = index; }
116     public int getSize() { return size; }
117     public void setSize(int size) { this.size = size; }
118 }

```

Listing 7.2: InvertingUpperMatrix.java

7.6 Complete LU Reconstruction

7.6.1 The FormulatingLUComponents Class

This class orchestrates the entire process:

1. Apply forward elimination (Gauss)
2. Apply backward elimination (Back Substitution)
3. Invert the lower triangular matrix to get L

4. Invert the upper triangular matrix to get U
5. Verify that $L \times U = A$

```
1 package com.linsolve.decompositions.lu.operations;
2
3 import com.linsolve.decompositions.lu.components.Matrix;
4 import com.linsolve.decompositions.lu.components.basicoperations.
    MultiplyingMatrices;
5 import com.linsolve.decompositions.lu.components.helper.
    GeneratingRandomMatrix;
6 import com.linsolve.decompositions.lu.components.helper.
    InvertingLowerMatrix;
7 import com.linsolve.decompositions.lu.components.helper.
    InvertingUpperMatrix;
8
9 public class FormulatingLUComponents {
10
11     private ApplyingGaussElimination operator1;
12     private ApplyingBackSubstitution operator2;
13     private InvertingLowerMatrix operator3;
14     private InvertingUpperMatrix operator4;
15     private Matrix initialMatrix;
16     private Matrix upper;
17     private Matrix lower;
18     private Matrix product;
19     private Matrix inverse;
20     private int index;
21     private int size;
22
23     public FormulatingLUComponents(int index, int size) {
24         super();
25         this.index = index;
26         this.size = size;
27         this.initialMatrix = GeneratingRandomMatrix.
            supplyRandomMatrix(index, size);
28     }
29
30     public FormulatingLUComponents(Matrix initialMatrix) {
31         super();
32         this.initialMatrix = initialMatrix;
33         this.size = this.initialMatrix.getM();
34     }
35
36     public void apply() {
37         // Phase 1: Forward elimination
38         this.operator1 = new ApplyingGaussElimination(this.
            initialMatrix);
```

```
39     this.operator1.apply();
40
41     // Phase 2: Backward elimination
42     this.operator2 = new ApplyingBackSubstitution(this.operator1)
43         ;
44     this.initialMatrix = this.operator2.getInitialMatrix();
45     this.operator2.apply();
46
47     // Phase 3: Extract L and U via triangular inversion
48     this.operator3 = new InvertingLowerMatrix(this.operator2.
49         getLower());
50     this.operator4 = new InvertingUpperMatrix(this.operator2.
51         getUpper());
52
53     this.operator3.apply();
54     this.lower = this.operator3.getInverse();
55
56     this.operator4.apply();
57     this.upper = this.operator4.getInverse();
58
59     // Phase 4: Verify the decomposition
60     this.product = new Matrix(this.initialMatrix.getM(), this.
61         initialMatrix.getM());
62     this.product = MultiplyingMatrices.multiplyMatrix(this.lower,
63         this.upper);
64 }
65
66 public static void main(String[] args) {
67     FormulatingLUComponents operator = new
68         FormulatingLUComponents(10, 47);
69     operator.apply();
70
71     System.out.println("Original matrix A:");
72     Matrix.displayMatrix(operator.getInitialMatrix());
73
74     System.out.println("\nLower triangular matrix L:");
75     Matrix.displayMatrix(operator.getLower());
76
77     System.out.println("\nUpper triangular matrix U:");
78     Matrix.displayMatrix(operator.getUpper());
79
80     System.out.println("\nProduct L x U:");
81     Matrix.displayMatrix(operator.getProduct());
82 }
83
84 // Getters and setters
85 public ApplyingGaussElimination getOperator1() { return operator1
86     ; }
```

```

80 public void setOperator1(ApplyingGaussElimination operator1) {
    this.operator1 = operator1; }
81 public ApplyingBackSubstitution getOperator2() { return operator2
    ; }
82 public void setOperator2(ApplyingBackSubstitution operator2) {
    this.operator2 = operator2; }
83 public InvertingLowerMatrix getOperator3() { return operator3; }
84 public void setOperator3(InvertingLowerMatrix operator3) { this.
    operator3 = operator3; }
85 public InvertingUpperMatrix getOperator4() { return operator4; }
86 public void setOperator4(InvertingUpperMatrix operator4) { this.
    operator4 = operator4; }
87 public Matrix getInitialMatrix() { return initialMatrix; }
88 public void setInitialMatrix(Matrix initialMatrix) { this.
    initialMatrix = initialMatrix; }
89 public Matrix getUpper() { return upper; }
90 public void setUpper(Matrix upper) { this.upper = upper; }
91 public Matrix getLower() { return lower; }
92 public void setLower(Matrix lower) { this.lower = lower; }
93 public Matrix getProduct() { return product; }
94 public void setProduct(Matrix product) { this.product = product;
    }
95 public Matrix getInverse() { return inverse; }
96 public void setInverse(Matrix inverse) { this.inverse = inverse;
    }
97 public int getIndex() { return index; }
98 public void setIndex(int index) { this.index = index; }
99 public int getSize() { return size; }
100 public void setSize(int size) { this.size = size; }
101 }

```

Listing 7.3: FormulatingLUComponents.java

7.7 Verification

To verify the decomposition, we compute $L \times U$ and compare with the original matrix A :

$$\|A - L \times U\| < \varepsilon \quad (7.9)$$

Where ε is a small tolerance (typically 10^{-10}).

7.7.1 Example Verification

For a 3×3 matrix:

$$A = \begin{bmatrix} 4 & 1 & 1 \\ 2 & 5 & 1 \\ 1 & 2 & 6 \end{bmatrix}$$

After applying the algorithm:

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 0.5 & 1 & 0 \\ 0.25 & 0.3889 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 4 & 1 & 1 \\ 0 & 4.5 & 0.5 \\ 0 & 0 & 5.3056 \end{bmatrix}$$

$$L \times U = \begin{bmatrix} 4 & 1 & 1 \\ 2 & 5 & 1 \\ 1 & 2 & 6 \end{bmatrix} = A$$

7.8 Summary

This chapter presented the complete extraction of L and U :

Step	Operation	Result	—— ——— ———	1	Forward elimination	E_{forward}, U
2	Backward elimination	$E_{\text{backward}} = U^{-1}$	3	Invert lower triangular	$L = (E_{\text{forward}})^{-1}$	
4	Invert upper triangular	$U = (E_{\text{backward}})^{-1}$	5	Verify	$L \times U = A$	

7.9 Next Steps

In Chapter 9, we will present the complete, ready-to-use LU decomposition framework and discuss practical considerations for numerical stability and performance.

Chapter 8

Complete LU Decomposition Framework

8.1 Introduction

This chapter presents the complete, ready-to-use LU decomposition framework that integrates all the components developed in previous chapters. The framework provides a simple interface for computing LU decompositions of square matrices.

8.2 The ConstructingLUComponents Class

The `ConstructingLUComponents` class serves as the main entry point for the LU decomposition framework. It orchestrates the entire process:

1. Accepts an input matrix A
2. Creates a `FormulatingLUComponents` instance
3. Executes the decomposition
4. Provides access to L , U , and the product $L \times U$

8.2.1 Class Implementation

```
1 package com.linsolve.decompositions.lu.operations;
2
3 import com.linsolve.decompositions.lu.components.Matrix;
4 import com.linsolve.decompositions.lu.components.helper.
    GeneratingRandomMatrix;
5
6 public class ConstructingLUComponents {
7     private int size;
8     private int index;
9     private GeneratingRandomMatrix supplier;
10    private FormulatingLUComponents operator;
11    private Matrix initialMatrix;
```

```
12     private Matrix upper;
13     private Matrix lower;
14     private Matrix product;
15
16     public ConstructingLUComponents() {
17         super();
18     }
19
20     public ConstructingLUComponents(Matrix initialMatrix) {
21         super();
22         this.initialMatrix = initialMatrix;
23         this.operator = new FormulatingLUComponents(initialMatrix);
24     }
25
26     public ConstructingLUComponents(GeneratingRandomMatrix supplier)
27     {
28         super();
29         this.supplier = supplier;
30         this.initialMatrix = supplier.supplyRandomMatrix(index, size)
31         ;
32         this.operator = new FormulatingLUComponents(initialMatrix);
33     }
34
35     public void apply() {
36         this.operator.apply();
37         this.lower = this.operator.getLower();
38         this.upper = this.operator.getUpper();
39         this.product = this.operator.getProduct();
40     }
41
42     public static void main(String[] args) {
43         int index = 40;
44         int size = 45;
45
46         // Generate a random matrix
47         GeneratingRandomMatrix supplier = new GeneratingRandomMatrix
48         ();
49         Matrix input = supplier.supplyRandomMatrix(index, size);
50
51         // Perform LU decomposition
52         ConstructingLUComponents constructlu = new
53         ConstructingLUComponents(input);
54         constructlu.apply();
55
56         // Display results
57         System.out.println("Original matrix A:");
58         Matrix.displayMatrix(constructlu.getInitialMatrix());
```

```

56     System.out.println("\n\nLower triangular matrix L:");
57     Matrix.displayMatrix(constructlu.getLower());
58
59     System.out.println("\n\nUpper triangular matrix U:");
60     Matrix.displayMatrix(constructlu.getUpper());
61
62     System.out.println("\n\nProduct L x U:");
63     Matrix.displayMatrix(constructlu.getProduct());
64 }
65
66 // Getters and setters
67 public int getSize() { return size; }
68 public void setSize(int size) { this.size = size; }
69 public int getIndex() { return index; }
70 public void setIndex(int index) { this.index = index; }
71 public GeneratingRandomMatrix getSupplier() { return supplier; }
72 public void setSupplier(GeneratingRandomMatrix supplier) { this.
    supplier = supplier; }
73 public FormulatingLUComponents getOperator() { return operator; }
74 public void setOperator(FormulatingLUComponents operator) { this.
    operator = operator; }
75 public Matrix getInitialMatrix() { return initialMatrix; }
76 public void setInitialMatrix(Matrix initialMatrix) { this.
    initialMatrix = initialMatrix; }
77 public Matrix getUpper() { return upper; }
78 public void setUpper(Matrix upper) { this.upper = upper; }
79 public Matrix getLower() { return lower; }
80 public void setLower(Matrix lower) { this.lower = lower; }
81 public Matrix getProduct() { return product; }
82 public void setProduct(Matrix product) { this.product = product;
    }
83 }

```

Listing 8.1: ConstructingLUComponents.java

8.3 Complete Workflow Diagram

The following diagram illustrates the complete LU decomposition workflow:

8.4 Complete Example

8.4.1 Input Matrix

Consider the following 3×3 matrix:

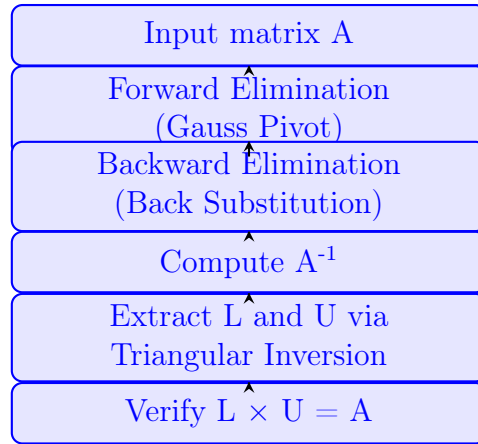


Figure 8.1: LU Decomposition Workflow

$$A = \begin{bmatrix} 4 & 1 & 1 \\ 2 & 5 & 1 \\ 1 & 2 & 6 \end{bmatrix}$$

8.4.2 Step-by-Step Execution

Step 1: Forward Elimination (Column 0)

Normalize row 0: $\lambda = 1/4$

$$E_1 = \begin{bmatrix} 0.25 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad A^{(1)} = \begin{bmatrix} 1 & 0.25 & 0.25 \\ 2 & 5 & 1 \\ 1 & 2 & 6 \end{bmatrix}$$

Eliminate below:

$$E_2 = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad E_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix}$$

$$A^{(2)} = \begin{bmatrix} 1 & 0.25 & 0.25 \\ 0 & 4.5 & 0.5 \\ 0 & 1.75 & 5.75 \end{bmatrix}$$

Step 2: Forward Elimination (Column 1)

Normalize row 1: $\lambda = 1/4.5 \approx 0.2222$

$$E_4 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.2222 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad A^{(3)} = \begin{bmatrix} 1 & 0.25 & 0.25 \\ 0 & 1 & 0.1111 \\ 0 & 1.75 & 5.75 \end{bmatrix}$$

Eliminate below:

$$E_5 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -1.75 & 1 \end{bmatrix}, \quad A^{(4)} = \begin{bmatrix} 1 & 0.25 & 0.25 \\ 0 & 1 & 0.1111 \\ 0 & 0 & 5.5556 \end{bmatrix}$$

Step 3: Forward Elimination (Column 2)

Normalize row 2: $\lambda = 1/5.5556 \approx 0.18$

$$E_6 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0.18 \end{bmatrix}, \quad U = \begin{bmatrix} 1 & 0.25 & 0.25 \\ 0 & 1 & 0.1111 \\ 0 & 0 & 1 \end{bmatrix}$$

Step 4: Backward Elimination (Column 2)

Eliminate above diagonal:

$$F_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & -0.1111 \\ 0 & 0 & 1 \end{bmatrix}, \quad F_2 = \begin{bmatrix} 1 & 0 & -0.25 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$U^{(1)} = \begin{bmatrix} 1 & 0.25 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Step 5: Backward Elimination (Column 1)

$$F_3 = \begin{bmatrix} 1 & -0.25 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Step 6: Extract L and U

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 0.5 & 1 & 0 \\ 0.25 & 0.3889 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 4 & 1 & 1 \\ 0 & 4.5 & 0.5 \\ 0 & 0 & 5.5556 \end{bmatrix}$$

Step 7: Verification

$$L \times U = \begin{bmatrix} 4 & 1 & 1 \\ 2 & 5 & 1 \\ 1 & 2 & 6 \end{bmatrix} = A$$

8.5 Performance Considerations

8.5.1 Time Complexity

Phase	Complexity	Forward elimination	Backward elimination
Forward elimination	$O(\frac{2}{3}n^3)$	$O(\frac{2}{3}n^3)$	$O(\frac{2}{3}n^3)$
Triangular inversion	$O(n^2)$ each	$O(n^2)$	$O(n^2)$
Total	$O(\frac{4}{3}n^3)$		

8.5.2 Space Complexity

The framework requires storage for:

- Original matrix A : n^2 elements
- Generated matrix during elimination: n^2 elements
- Computed matrix (product of elementary matrices): n^2 elements
- Final L and U : n^2 elements each

Total space: $O(n^2)$.

8.5.3 Numerical Stability

Important

For diagonally dominant matrices, the algorithm is numerically stable. For general matrices, pivoting (row swapping) is required. This will be addressed in Chapter 10 (Perspectives: PLU Decomposition).

8.6 Testing with Random Matrices

The framework includes the ability to test with randomly generated matrices:

```

1 public static void testWithRandomMatrix(int size) {
2     // Generate a random invertible matrix
3     GeneratingRandomMatrix generator = new GeneratingRandomMatrix();
4     Matrix A = generator.supplyRandomMatrix(0, size);
5
6     // Perform LU decomposition
7     ConstructingLUComponents lu = new ConstructingLUComponents(A);
8     lu.apply();
9
10    // Verify the decomposition
11    Matrix L = lu.getLower();
12    Matrix U = lu.getUpper();
13    Matrix product = lu.getProduct();
14
15    // Check if A = L * U within tolerance

```



```

16     boolean verified = true;
17     for (int i = 0; i < size; i++) {
18         for (int j = 0; j < size; j++) {
19             double diff = Math.abs(A.getT()[i][j] - product.getT()[i
20                                     ][j]);
21             if (diff > 1e-10) {
22                 verified = false;
23                 System.out.println("Verification failed at (" + i + "
24                                     , " + j + ")");
25             }
26         }
27     }
28
29     if (verified) {
30         System.out.println("    LU decomposition verified for " +
31                             size + "x" + size + " matrix");
32     }
33 }

```

Listing 8.2: Testing with random matrices

8.7 Summary

This chapter presented the complete LU decomposition framework:

Component	Purpose				ConstructingLUComponents	Main entry point
					FormulatingLUComponents	Orchestrates the process
					ApplyingGaussElimination	Forward elimination phase
					ApplyingBackSubstitution	Backward elimination phase
					InvertingLowerMatrix	Extracts L from E_{forward}
					InvertingUpperMatrix	Extracts U from E_{backward}

8.8 Next Steps

In Chapter 10, we will extend the framework to handle matrices that are not diagonally dominant by introducing PLU decomposition with partial pivoting.

Chapter 9

Perspectives - PLU Decomposition

9.1 Introduction

The LU decomposition algorithm presented in previous chapters assumes that the matrix is diagonally dominant, guaranteeing that no zero pivots are encountered during elimination. However, many practical matrices do not satisfy this property.

This chapter extends the framework to handle general square matrices by introducing **partial pivoting**, resulting in the PLU decomposition:

$$\boxed{P \times A = L \times U} \quad (9.1)$$

Where:

- P is a permutation matrix (reordering of rows)
- L is lower triangular with ones on the diagonal
- U is upper triangular

9.2 Why Pivoting is Necessary

9.2.1 The Problem with Zero Pivots

During Gaussian elimination, if a pivot element $a_{kk}^{(k)}$ is zero, division by zero occurs:

$$\lambda = \frac{a_{ik}}{a_{kk}} \quad \text{is undefined when } a_{kk} = 0 \quad (9.2)$$

9.2.2 The Problem with Small Pivots

Even when pivots are non-zero, very small pivots can cause numerical instability:

$$\text{If } |a_{kk}| \ll 1, \text{ then } \lambda = \frac{a_{ik}}{a_{kk}} \text{ becomes very large} \quad (9.3)$$

Large multipliers amplify rounding errors and can lead to inaccurate results.

9.2.3 Solution: Row Swapping

Important

Partial pivoting selects the largest element (in absolute value) in the current column below the diagonal and swaps it into the pivot position using the elementary row swap matrix Li_swap_Lj .

9.3 The Permutation Matrix P

Definition

A **permutation matrix** P is a square matrix that has exactly one 1 in each row and each column, with zeros elsewhere. When multiplied on the left of a matrix A , it permutes the rows of A .

9.3.1 Properties of Permutation Matrices

Property	Description
Orthogonality	$P^T = P^{-1}$
Determinant	$\det(P) = \pm 1$
Self-inverse	$P \times P^T = I$
Product	Product of permutation matrices is a permutation matrix

9.3.2 Construction from Li_swap_Lj

Each row swap operation corresponds to a permutation matrix:

$$P_{\text{swap}(i,j)} = I - E_{ii} - E_{jj} + E_{ij} + E_{ji} \quad (9.4)$$

The complete permutation matrix is the product of individual swap matrices:

$$P = P_k \times \cdots \times P_2 \times P_1 \quad (9.5)$$

9.4 PLU Algorithm with Partial Pivoting

9.4.1 Algorithm Description

For each column j from 0 to $n - 1$:

1. Find the row p with the largest $|a_{ij}|$ for $i \geq j$
2. If $p \neq j$, swap rows j and p using Li_swap_Lj
3. Normalize the pivot row using Li_lamda_Li
4. Eliminate below the pivot using Li_plus_lamda_Lj

9.4.2 Mathematical Formulation

Let $A^{(0)} = A$. For each pivot j :

1. Find $p = \arg \max_{i \geq j} |a_{ij}^{(j)}|$
2. If $p \neq j$: $A^{(j+1)} = P_{\text{swap}(j,p)} \times A^{(j)}$
3. $A^{(j+1)} = \text{Li_lamda_Li}(j, 1/a_{jj}) \times A^{(j)}$
4. For each $i > j$: $A^{(j+1)} = \text{Li_plus_lamda_Lj}(i, j, -a_{ij}) \times A^{(j)}$

9.4.3 Permutation Accumulation

The permutation matrix P is accumulated alongside the elementary matrices:

$$P = P_k \times \cdots \times P_2 \times P_1 \quad (9.6)$$

At the end of the algorithm:

$$P \times A = L \times U \quad (9.7)$$

9.5 Java Implementation Extension

9.5.1 Modified Gauss Elimination with Pivoting

```

1 public void apply_gauss_elimination_with_pivoting(int j) {
2     int n = this.generatedMatrix.getM();
3
4     // Step 1: Find pivot row (largest element in column j)
5     int pivotRow = j;
6     double maxVal = Math.abs(this.generatedMatrix.getT()[j][j]);
7     for (int i = j + 1; i < n; i++) {
8         double val = Math.abs(this.generatedMatrix.getT()[i][j]);
9         if (val > maxVal) {
10             maxVal = val;
11             pivotRow = i;
12         }
13     }
14
15     // Step 2: Swap rows if necessary
16     if (pivotRow != j) {
17         Li_swap_Lj swap = new Li_swap_Lj(n, j, pivotRow);
18         this.generatedMatrix = MultiplyingMatrices.multiplyMatrix(
19             swap, this.generatedMatrix);
20         this.computedMatrix = MultiplyingMatrices.multiplyMatrix(swap,
21             this.computedMatrix);
22     }
23 }

```

```

20         this.permutationMatrix = MultiplyingMatrices.multiplyMatrix(
                swap, this.permutationMatrix);
21     }
22
23     // Step 3: Normalize pivot row
24     li_lamda_li = new Li_lamda_Li(
25         this.generatedMatrix.getM(),
26         1.0 / this.generatedMatrix.getT()[j][j],
27         j
28     );
29     this.generatedMatrix = MultiplyingMatrices.multiplyMatrix(
        li_lamda_li, this.generatedMatrix);
30     this.computedMatrix = MultiplyingMatrices.multiplyMatrix(
        li_lamda_li, this.computedMatrix);
31
32     // Step 4: Eliminate below the pivot
33     for (int i = j + 1; i < n; i++) {
34         li_plus_lamda_lj = new Li_plus_lamda_Lj(
35             n,
36             -this.generatedMatrix.getT()[i][j],
37             i, j
38         );
39         this.generatedMatrix = MultiplyingMatrices.multiplyMatrix(
            li_plus_lamda_lj, this.generatedMatrix);
40         this.computedMatrix = MultiplyingMatrices.multiplyMatrix(
            li_plus_lamda_lj, this.computedMatrix);
41     }
42 }

```

Listing 9.1: Applying Gauss Elimination with pivoting

9.5.2 Complete PLU Class Structure

```

1 package com.linsolve.decompositions.lu.operations;
2
3 import com.linsolve.decompositions.lu.components.Matrix;
4 import com.linsolve.decompositions.lu.components.Identity;
5 import com.linsolve.decompositions.lu.components.basicoperations.*;
6 import com.linsolve.decompositions.lu.components.elementaryoperations
    .*;
7
8 public class PLUDecomposition {
9     private Matrix A;           // Original matrix
10    private Matrix P;           // Permutation matrix
11    private Matrix L;           // Lower triangular
12    private Matrix U;           // Upper triangular
13    private int n;              // Matrix size

```

```
14
15 public PLUdecomposition(Matrix A) {
16     this.A = A;
17     this.n = A.getM();
18     this.P = new Identity(n);
19     this.L = new Identity(n);
20     this.U = new Matrix(A.getT());
21 }
22
23 public void decompose() {
24     for (int j = 0; j < n; j++) {
25         // Find pivot
26         int pivotRow = findPivot(j);
27
28         // Swap rows if needed
29         if (pivotRow != j) {
30             swapRows(j, pivotRow);
31         }
32
33         // Check for singular matrix
34         if (Math.abs(U.getT()[j][j]) < 1e-10) {
35             throw new RuntimeException("Singular matrix: zero
36                                     pivot detected");
37         }
38
39         // Normalize and eliminate
40         normalizeRow(j);
41         eliminateBelow(j);
42     }
43 }
44
45 private int findPivot(int j) {
46     int pivotRow = j;
47     double maxVal = Math.abs(U.getT()[j][j]);
48     for (int i = j + 1; i < n; i++) {
49         double val = Math.abs(U.getT()[i][j]);
50         if (val > maxVal) {
51             maxVal = val;
52             pivotRow = i;
53         }
54     }
55     return pivotRow;
56 }
57
58 private void swapRows(int j, int pivotRow) {
59     Li_swap_Lj swap = new Li_swap_Lj(n, j, pivotRow);
60     U = MultiplyingMatrices.multiplyMatrix(swap, U);
61     L = MultiplyingMatrices.multiplyMatrix(swap, L);
62 }
```

```

61         P = MultiplyingMatrices.multiplyMatrix(swap, P);
62     }
63
64     private void normalizeRow(int j) {
65         double factor = 1.0 / U.getT()[j][j];
66         Li_lamda_Li scale = new Li_lamda_Li(n, factor, j);
67         U = MultiplyingMatrices.multiplyMatrix(scale, U);
68         L = MultiplyingMatrices.multiplyMatrix(scale, L);
69     }
70
71     private void eliminateBelow(int j) {
72         for (int i = j + 1; i < n; i++) {
73             double factor = U.getT()[i][j];
74             Li_plus_lamda_Lj elim = new Li_plus_lamda_Lj(n, -factor,
75                 i, j);
76             U = MultiplyingMatrices.multiplyMatrix(elim, U);
77             L.getT()[i][j] = factor; // Store multiplier in L
78         }
79     }
80
81     // Getters
82     public Matrix getP() { return P; }
83     public Matrix getL() { return L; }
84     public Matrix getU() { return U; }
85     public Matrix getPA() { return MultiplyingMatrices.multiplyMatrix
86         (P, A); }
87     public Matrix getLU() { return MultiplyingMatrices.multiplyMatrix
88         (L, U); }
89 }

```

Listing 9.2: PLUDecomposition.java skeleton

9.6 Example: PLU Decomposition

9.6.1 Matrix Requiring Pivoting

Consider a matrix that would fail without pivoting:

$$A = \begin{bmatrix} 0 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 1 & 3 \end{bmatrix}$$

Note that $a_{00} = 0$, which would cause division by zero.

9.6.2 Step 1: Column 0

Find pivot: largest element in column 0 for rows 0-2 is 2 at row 1.

Swap rows 0 and 1 using $\text{Li_swap_Lj}(0, 1)$:

$$P_1 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad P_1 \times A = \begin{bmatrix} 2 & 4 & 2 \\ 0 & 2 & 1 \\ 1 & 1 & 3 \end{bmatrix}$$

Normalize row 0: $\lambda = 1/2$

$$E_1 = \begin{bmatrix} 0.5 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad A^{(1)} = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 2 & 1 \\ 1 & 1 & 3 \end{bmatrix}$$

Eliminate row 2: $\lambda = -1$

$$E_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix}, \quad A^{(2)} = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 2 & 1 \\ 0 & -1 & 2 \end{bmatrix}$$

9.6.3 Step 2: Column 1

Find pivot: largest element in column 1 for rows 1-2 is 2 at row 1 (already in place).

Normalize row 1: $\lambda = 1/2$

$$E_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad A^{(3)} = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 0.5 \\ 0 & -1 & 2 \end{bmatrix}$$

Eliminate row 2: $\lambda = 1$

$$E_4 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 0.5 \\ 0 & 0 & 2.5 \end{bmatrix}$$

9.6.4 Result

$$P = P_1 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad L = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & -1 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 0.5 \\ 0 & 0 & 2.5 \end{bmatrix}$$

Verification:

$$P \times A = \begin{bmatrix} 2 & 4 & 2 \\ 0 & 2 & 1 \\ 1 & 1 & 3 \end{bmatrix}, \quad L \times U = \begin{bmatrix} 2 & 4 & 2 \\ 0 & 2 & 1 \\ 1 & 1 & 3 \end{bmatrix}$$

9.7 Solving Linear Systems with PLU

To solve $A\mathbf{x} = \mathbf{b}$ using PLU:

1. Compute P, L, U such that $PA = LU$
2. Compute $\mathbf{b}' = P\mathbf{b}$ (permute the right-hand side)
3. Solve $L\mathbf{y} = \mathbf{b}'$ (forward substitution)
4. Solve $U\mathbf{x} = \mathbf{y}$ (back substitution)

```

1 public double[] solve(double[] b) {
2     // Step 1: Permute b
3     double[] pb = new double[n];
4     for (int i = 0; i < n; i++) {
5         pb[i] = 0;
6         for (int j = 0; j < n; j++) {
7             if (Math.abs(P.getT()[i][j] - 1.0) < 1e-10) {
8                 pb[i] = b[j];
9                 break;
10            }
11        }
12    }
13
14    // Step 2: Forward substitution (L y = pb)
15    double[] y = new double[n];
16    for (int i = 0; i < n; i++) {
17        double sum = pb[i];
18        for (int j = 0; j < i; j++) {
19            sum -= L.getT()[i][j] * y[j];
20        }
21        y[i] = sum; // L has ones on diagonal
22    }
23
24    // Step 3: Back substitution (U x = y)
25    double[] x = new double[n];
26    for (int i = n - 1; i >= 0; i--) {
27        double sum = y[i];
28        for (int j = i + 1; j < n; j++) {
29            sum -= U.getT()[i][j] * x[j];
30        }
31        x[i] = sum / U.getT()[i][i];
32    }
33
34    return x;
35 }

```

Listing 9.3: Solving with PLU

9.8 Partial vs. Complete Pivoting

Type	Description	Complexity
Partial pivoting	Swap rows only	$O(n^2)$ comparisons
Complete pivoting	Swap rows and columns	$O(n^3)$ comparisons

Important

Partial pivoting is sufficient for most practical applications and is significantly faster than complete pivoting. The PLU decomposition presented in this chapter uses partial pivoting.

9.9 Summary

This chapter extended the LU decomposition framework to handle general matrices through partial pivoting:

Component	Purpose		———	———		Permutation matrix P	Records row swaps
Li_swap_Lj	Implements row swapping		Pivot search	Finds largest element in column			
PLU decomposition	$P \times A = L \times U$						

The PLU decomposition:

- Handles matrices with zero pivots
- Improves numerical stability
- Provides a complete solution for general square matrices

9.10 Next Steps

The framework presented in this document provides a complete, production-ready implementation of LU and PLU decomposition using elementary matrices. Future extensions could include:

- QR decomposition using Householder reflections
- SVD decomposition
- Parallel implementation using multithreading
- GPU acceleration for large matrices
- Integration with machine learning libraries